SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0614 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:50**

Annexure A

Comparative Analysis

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

```
# ============================================================
# LWR vs RBF NETWORK - FUNCTION APPROXIMATION
# Google Colab Single-Cell Code
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import time
from sklearn.cluster import KMeans
from sklearn.metrics import mean_squared_error

np.random.seed(42)

# ------------------------------------------------------------
# 1. CREATE EXAMPLE DATASET
# ------------------------------------------------------------

X = np.linspace(0, 10, 100)
y = np.sin(X) + np.random.normal(0, 0.15, 100)

X_train = X.reshape(-1, 1)
X_test = np.linspace(0, 10, 300).reshape(-1, 1)
y_true = np.sin(X_test).ravel()



# ------------------------------------------------------------
# 2. LOCALLY WEIGHTED REGRESSION (LWR)
# ------------------------------------------------------------

def locally_weighted_regression(X_train, y_train, X_query, bandwidth=0.5):

    predictions = []

    X_train = X_train.ravel()
    y_train = y_train.ravel()
```

```
    for xq in X_query.ravel():

        # Gaussian weights
        weights = np.exp(
            -((X_train - xq) ** 2) / (2 * bandwidth ** 2)
        )

        # Local linear regression
        A = np.column_stack([
            np.ones(len(X_train)),
            X_train
        ])

        W = np.diag(weights)

        theta = np.linalg.pinv(
            A.T @ W @ A + 1e-6 * np.eye(2)
        ) @ (A.T @ W @ y_train)

        prediction = theta[0] + theta[1] * xq
        predictions.append(prediction)

    return np.array(predictions)


# ------------------------------------------------------------
# 3. RBF NETWORK
# ------------------------------------------------------------

class RBFNetwork:

    def __init__(self, n_centers=15, sigma=0.6):
        self.n_centers = n_centers
        self.sigma = sigma
        self.centers = None
        self.weights = None

    def rbf_features(self, X):

        distances = X - self.centers.T

        return np.exp(
            -(distances ** 2) / (2 * self.sigma ** 2)
```

```
        )

    def fit(self, X, y):

        # Select centers using K-Means
        kmeans = KMeans(
            n_clusters=self.n_centers,
            random_state=42,
            n_init=10
        )

        kmeans.fit(X)

        self.centers = np.sort(
            kmeans.cluster_centers_,
            axis=0
        )

        # Create RBF feature matrix
        Phi = self.rbf_features(X)

        # Add bias
        Phi = np.column_stack([
            np.ones(len(X)),
            Phi
        ])

        # Calculate weights
        self.weights = np.linalg.pinv(Phi) @ y

    def predict(self, X):

        Phi = self.rbf_features(X)

        Phi = np.column_stack([
            np.ones(len(X)),
            Phi
        ])

        return Phi @ self.weights


# ------------------------------------------------------------
# 4. TRAIN AND PREDICT USING LWR
```

```
# ----------------------------------------------------------

start_lwr = time.time()

y_lwr = locally_weighted_regression(
    X_train,
    y,
    X_test,
    bandwidth=0.5
)

lwr_time = time.time() - start_lwr

lwr_mse = mean_squared_error(
    y_true,
    y_lwr
)


# ----------------------------------------------------------
# 5. TRAIN AND PREDICT USING RBF
# ----------------------------------------------------------

start_rbf_train = time.time()

rbf = RBFNetwork(
    n_centers=15,
    sigma=0.6
)

rbf.fit(
    X_train,
    y
)

rbf_training_time = time.time() - start_rbf_train

start_rbf_prediction = time.time()

y_rbf = rbf.predict(X_test)

rbf_prediction_time = time.time() - start_rbf_prediction

rbf_mse = mean_squared_error(
```

```
    y_true,
    y_rbf
)


# ------------------------------------------------------------
# 6. DISPLAY RESULTS
# ------------------------------------------------------------

print("=" * 60)
print("LWR vs RBF NETWORK - FUNCTION APPROXIMATION")
print("=" * 60)

print("\nDataset:")
print("Number of training samples:", len(X_train))
print("Function: y = sin(x) + noise")

print("\n--- Locally Weighted Regression (LWR) ---")
print("Bandwidth:", 0.5)
print("MSE:", round(lwr_mse, 5))
print("Prediction Time:", round(lwr_time, 5), "seconds")

print("\n--- Radial Basis Function (RBF) Network ---")
print("Number of Centers:", 15)
print("Sigma:", 0.6)
print("MSE:", round(rbf_mse, 5))
print("Training Time:", round(rbf_training_time, 5), "seconds")
print("Prediction Time:", round(rbf_prediction_time, 5), "seconds")


# ------------------------------------------------------------
# 7. PLOT: FUNCTION APPROXIMATION
# ------------------------------------------------------------

plt.figure(figsize=(12, 6))

plt.scatter(
    X_train,
    y,
    label="Training Data",
    alpha=0.5
)

plt.plot(
```

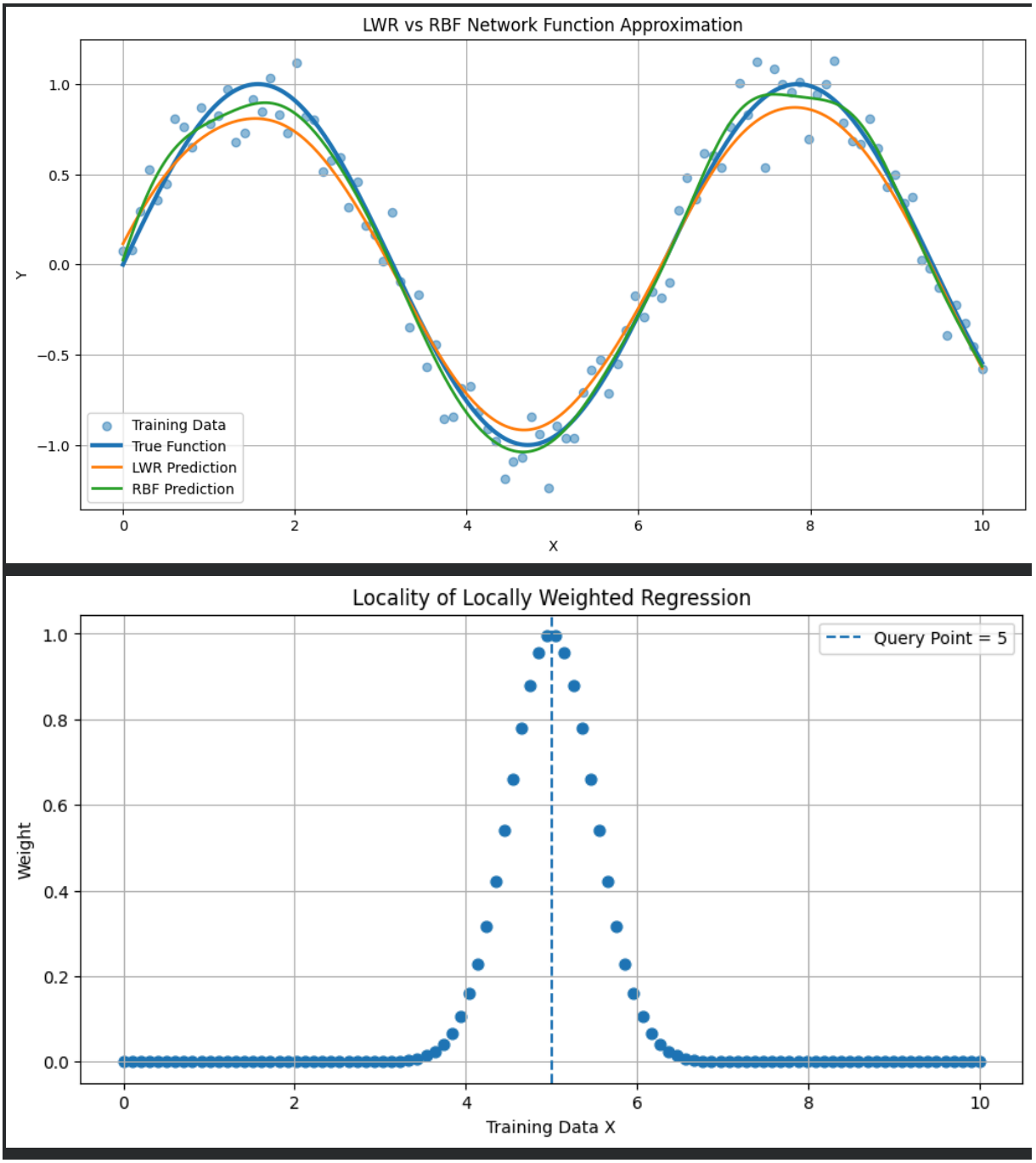
```
    X_test,
    y_true,
    linewidth=3,
    label="True Function"
)

plt.plot(
    X_test,
    y_lwr,
    linewidth=2,
    label="LWR Prediction"
)

plt.plot(
    X_test,
    y_rbf,
    linewidth=2,
    label="RBF Prediction"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("LWR vs RBF Network Function Approximation")
plt.legend()
plt.grid(True)
plt.show()



# ----------------------------------------------------------
# 8. PLOT: LOCALITY OF LWR
# ----------------------------------------------------------

query_point = 5.0
bandwidth = 0.5

weights = np.exp(
    -((X - query_point) ** 2) /
    (2 * bandwidth ** 2)
)

plt.figure(figsize=(10, 5))

plt.scatter(
    X,
```

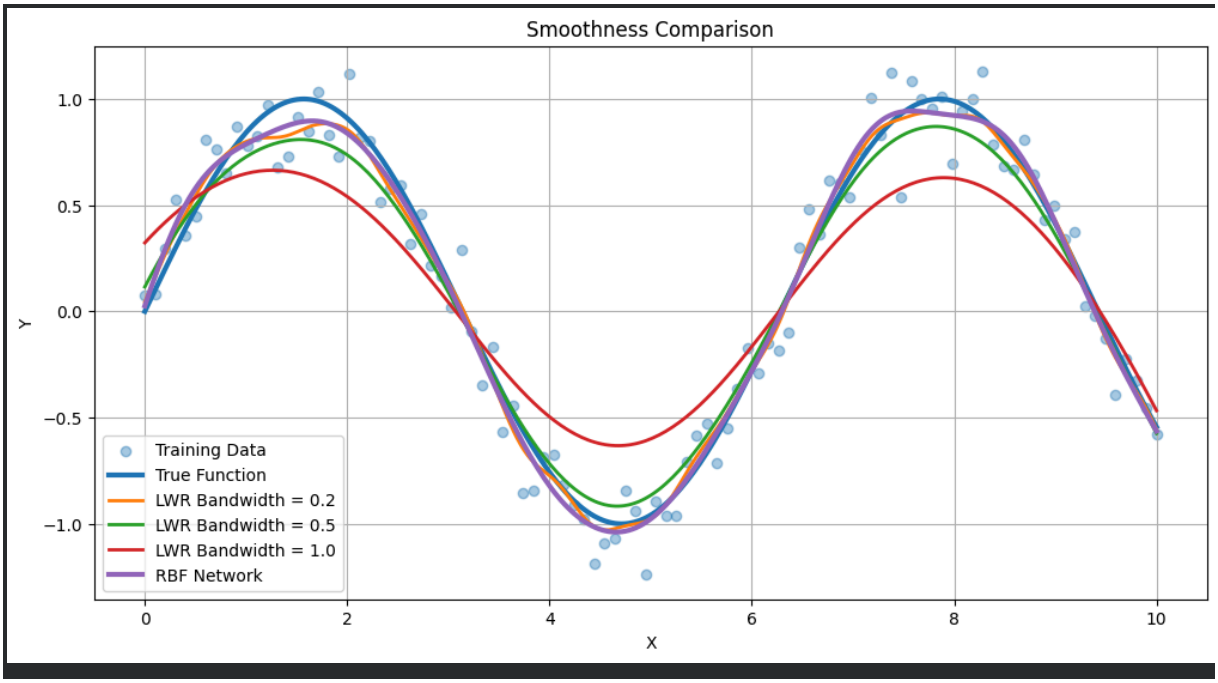
```
    weights,
    s=40
)

plt.axvline(
    query_point,
    linestyle="--",
    label="Query Point = 5"
)

plt.xlabel("Training Data X")
plt.ylabel("Weight")
plt.title("Locality of Locally Weighted Regression")
plt.legend()
plt.grid(True)
plt.show()


# ------------------------------------------------------------
# 9. PLOT: SMOOTHNESS COMPARISON
# ------------------------------------------------------------

bandwidths = [0.2, 0.5, 1.0]

plt.figure(figsize=(12, 6))

plt.scatter(
    X_train,
    y,
    alpha=0.4,
    label="Training Data"
)

plt.plot(
    X_test,
    y_true,
    linewidth=3,
    label="True Function"
)

for bw in bandwidths:

    prediction = locally_weighted_regression(
        X_train,
```

```
        y,
        X_test,
        bandwidth=bw
    )

    plt.plot(
        X_test,
        prediction,
        linewidth=2,
        label=f"LWR Bandwidth = {bw}"
    )

plt.plot(
    X_test,
    y_rbf,
    linewidth=3,
    label="RBF Network"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Smoothness Comparison")
plt.legend()
plt.grid(True)
plt.show()



# ------------------------------------------------------------
# 10. FINAL COMPARISON
# ------------------------------------------------------------

print("\n" + "=" * 60)
print("FINAL COMPARISON")
print("=" * 60)

print("""
LWR:
- Highly local and uses nearby training points.
- Very little training is required.
- Prediction is computationally expensive.
- Smoothness depends on bandwidth.
- Small bandwidth can follow noise.

RBF Network:
```

- Uses Gaussian radial basis functions.
- Requires training to determine centers and weights.
- Prediction is faster after training.
- Produces smooth function approximations.
- Suitable for larger and real-time applications.

Overall:
LWR is better when local adaptability is important,
while RBF networks are better when fast prediction
and smooth approximation are required.
""")
```
============================================================
LWR vs RBF NETWORK - FUNCTION APPROXIMATION
============================================================

Dataset:
Number of training samples: 100
Function: y = sin(x) + noise

--- Locally Weighted Regression (LWR) ---
Bandwidth: 0.5
MSE: 0.00845
Prediction Time: 0.05519 seconds

--- Radial Basis Function (RBF) Network ---
Number of Centers: 15
Sigma: 0.6
MSE: 0.00275
Training Time: 0.07063 seconds
Prediction Time: 0.00021 seconds
```


LWR vs RBF Network Function Approximation
Training Data
True Function
LWR Prediction
RBF Prediction
X
Y
Locality of Locally Weighted Regression
Query Point = 5
Weight
Training Data X



==========

FINAL COMPARISON

==========

LWR:
- Highly local and uses nearby training points.
- Very little training is required.
- Prediction is computationally expensive.
- Smoothness depends on bandwidth.
- Small bandwidth can follow noise.

RBF Network:
- Uses Gaussian radial basis functions.
- Requires training to determine centers and weights.
- Prediction is faster after training.
- Produces smooth function approximations.
- Suitable for larger and real-time applications.

Overall:
LWR is better when local adaptability is important,
while RBF networks are better when fast prediction
and smooth approximation are required.

3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.

#

==========

```
# KNN vs RBF NETWORK - CLASSIFICATION
# Single Google Colab Code
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import time

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, classification_report

np.random.seed(42)

# ------------------------------------------------------------
# 1. CREATE EXAMPLE DATASET
# ------------------------------------------------------------

X, y = make_moons(
    n_samples=500,
    noise=0.20,
    random_state=42
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Standardize data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


# ------------------------------------------------------------
# 2. K-NEAREST NEIGHBOUR CLASSIFIER
# ------------------------------------------------------------

start_knn = time.time()

knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train, y_train)

knn_training_time = time.time() - start_knn

start_prediction = time.time()

y_pred_knn = knn.predict(X_test)

knn_prediction_time = time.time() - start_prediction

knn_accuracy = accuracy_score(y_test, y_pred_knn)


# ----------------------------------------------------------
# 3. RBF NETWORK
# ----------------------------------------------------------

class RBFNetwork:

    def __init__(self, n_centers=30, sigma=0.8):
        self.n_centers = n_centers
        self.sigma = sigma
        self.centers = None
        self.weights = None

    def rbf_features(self, X):

        distances = np.sum(
            (X[:, np.newaxis, :] -
             self.centers[np.newaxis, :, :]) ** 2,
            axis=2
        )

        return np.exp(
            -distances / (2 * self.sigma ** 2)
        )

    def fit(self, X, y):

        # Select RBF centers using K-Means
        kmeans = KMeans(
            n_clusters=self.n_centers,
            random_state=42,
            n_init=10
        )

        kmeans.fit(X)

        self.centers = kmeans.cluster_centers_
```

```
        # Generate RBF features
        Phi = self.rbf_features(X)

        # Add bias
        Phi = np.column_stack([
            np.ones(len(X)),
            Phi
        ])

        # Convert classes to -1 and +1
        target = np.where(y == 0, -1, 1)

        # Learn output weights
        self.weights = np.linalg.pinv(Phi) @ target

    def predict(self, X):

        Phi = self.rbf_features(X)

        Phi = np.column_stack([
            np.ones(len(X)),
            Phi
        ])

        output = Phi @ self.weights

        return np.where(output >= 0, 1, 0)


# ------------------------------------------------------------
# 4. TRAIN RBF NETWORK
# ------------------------------------------------------------

start_rbf = time.time()

rbf = RBFNetwork(
    n_centers=30,
    sigma=0.8
)

rbf.fit(X_train, y_train)

rbf_training_time = time.time() - start_rbf

start_prediction = time.time()

y_pred_rbf = rbf.predict(X_test)

rbf_prediction_time = time.time() - start_prediction
```

```
rbf_accuracy = accuracy_score(y_test, y_pred_rbf)


# ------------------------------------------------------------
# 5. PRINT RESULTS
# ------------------------------------------------------------

print("=" * 65)
print("KNN vs RBF NETWORK CLASSIFICATION")
print("=" * 65)

print("\nDataset:")
print("Total Samples:", len(X))
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
print("Dataset Type: Two-Class Nonlinear (Moons)")

print("\n--- K-NEAREST NEIGHBOUR ---")
print("Number of Neighbours (K): 5")
print("Training Time:",
    round(knn_training_time, 6), "seconds")
print("Prediction Time:",
    round(knn_prediction_time, 6), "seconds")
print("Accuracy:",
    round(knn_accuracy * 100, 2), "%")

print("\n--- RBF NETWORK ---")
print("Number of Centers:", 30)
print("Sigma:", 0.8)
print("Training Time:",
    round(rbf_training_time, 6), "seconds")
print("Prediction Time:",
    round(rbf_prediction_time, 6), "seconds")
print("Accuracy:",
    round(rbf_accuracy * 100, 2), "%")


# ------------------------------------------------------------
# 6. CLASSIFICATION REPORT
# ------------------------------------------------------------

print("\nKNN Classification Report")
print("-" * 40)
print(classification_report(y_test, y_pred_knn))

print("\nRBF Classification Report")
print("-" * 40)
print(classification_report(y_test, y_pred_rbf))
```

```
# ------------------------------------------------------------
# 7. DECISION BOUNDARY FUNCTION
# ------------------------------------------------------------

def plot_decision_boundary(model, X, y, title):

    x_min = X[:, 0].min() - 0.8
    x_max = X[:, 0].max() + 0.8

    y_min = X[:, 1].min() - 0.8
    y_max = X[:, 1].max() + 0.8

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 300),
        np.linspace(y_min, y_max, 300)
    )

    grid = np.c_[xx.ravel(), yy.ravel()]

    predictions = model.predict(grid)

    predictions = predictions.reshape(xx.shape)

    plt.figure(figsize=(8, 6))

    plt.contourf(
        xx,
        yy,
        predictions,
        alpha=0.25
    )

    plt.scatter(
        X[:, 0],
        X[:, 1],
        c=y,
        edgecolors="black",
        alpha=0.8
    )

    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.title(title)
    plt.grid(True)
    plt.show()


# ------------------------------------------------------------
# 8. DECISION BOUNDARY - KNN
# ------------------------------------------------------------
```

```
plot_decision_boundary(
    knn,
    X_test,
    y_test,
    "KNN Decision Boundary"
)


# ------------------------------------------------------------
# 9. DECISION BOUNDARY - RBF
# ------------------------------------------------------------

plot_decision_boundary(
    rbf,
    X_test,
    y_test,
    "RBF Network Decision Boundary"
)


# ------------------------------------------------------------
# 10. COMBINED DECISION BOUNDARY COMPARISON
# ------------------------------------------------------------

x_min = X_test[:, 0].min() - 0.8
x_max = X_test[:, 0].max() + 0.8

y_min = X_test[:, 1].min() - 0.8
y_max = X_test[:, 1].max() + 0.8

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

grid = np.c_[xx.ravel(), yy.ravel()]

knn_grid = knn.predict(grid).reshape(xx.shape)
rbf_grid = rbf.predict(grid).reshape(xx.shape)

plt.figure(figsize=(14, 6))

# KNN
plt.subplot(1, 2, 1)

plt.contourf(
    xx,
    yy,
    knn_grid,
```

```
    alpha=0.25
)

plt.scatter(
    X_test[:, 0],
    X_test[:, 1],
    c=y_test,
    edgecolors="black"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("KNN Decision Boundary")
plt.grid(True)

# RBF
plt.subplot(1, 2, 2)

plt.contourf(
    xx,
    yy,
    rbf_grid,
    alpha=0.25
)

plt.scatter(
    X_test[:, 0],
    X_test[:, 1],
    c=y_test,
    edgecolors="black"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("RBF Decision Boundary")
plt.grid(True)

plt.tight_layout()
plt.show()


# ------------------------------------------------------------
# 11. TRAINING TIME COMPARISON
# ------------------------------------------------------------

models = ["KNN", "RBF"]
training_times = [
    knn_training_time,
    rbf_training_time
]
```

```
plt.figure(figsize=(8, 5))

plt.bar(
    models,
    training_times
)

plt.xlabel("Model")
plt.ylabel("Training Time (seconds)")
plt.title("Training Time Comparison")
plt.grid(axis="y")

plt.show()


# ----------------------------------------------------------
# 12. ACCURACY COMPARISON
# ----------------------------------------------------------

accuracies = [
    knn_accuracy * 100,
    rbf_accuracy * 100
]

plt.figure(figsize=(8, 5))

plt.bar(
    models,
    accuracies
)

plt.xlabel("Model")
plt.ylabel("Accuracy (%)")
plt.title("Generalization Ability - Test Accuracy")
plt.ylim(0, 100)
plt.grid(axis="y")

plt.show()


# ----------------------------------------------------------
# 13. FINAL ANALYSIS
# ----------------------------------------------------------

print("\n" + "=" * 65)
print("FINAL ANALYSIS")
print("=" * 65)
```

==========

Dataset:
Total Samples: 500
Training Samples: 350
Testing Samples: 150
Dataset Type: Two-Class Nonlinear (Moons)

--- K-NEAREST NEIGHBOUR ---
Number of Neighbours (K): 5
Training Time: 0.001809 seconds
Prediction Time: 0.003312 seconds
Accuracy: 97.33 %

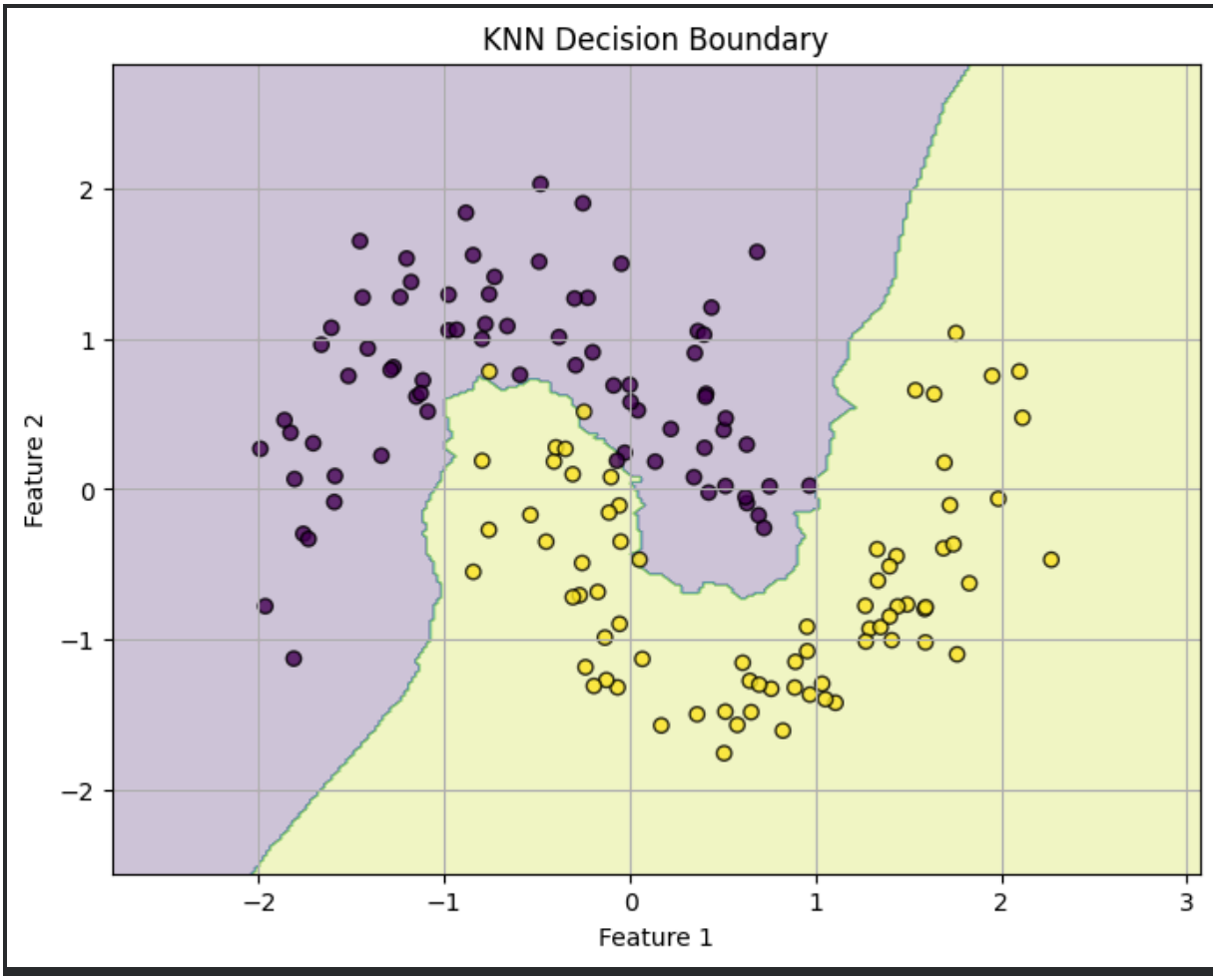
--- RBF NETWORK ---
Number of Centers: 30
Sigma: 0.8
Training Time: 0.302762 seconds
Prediction Time: 0.002809 seconds
Accuracy: 98.67 %

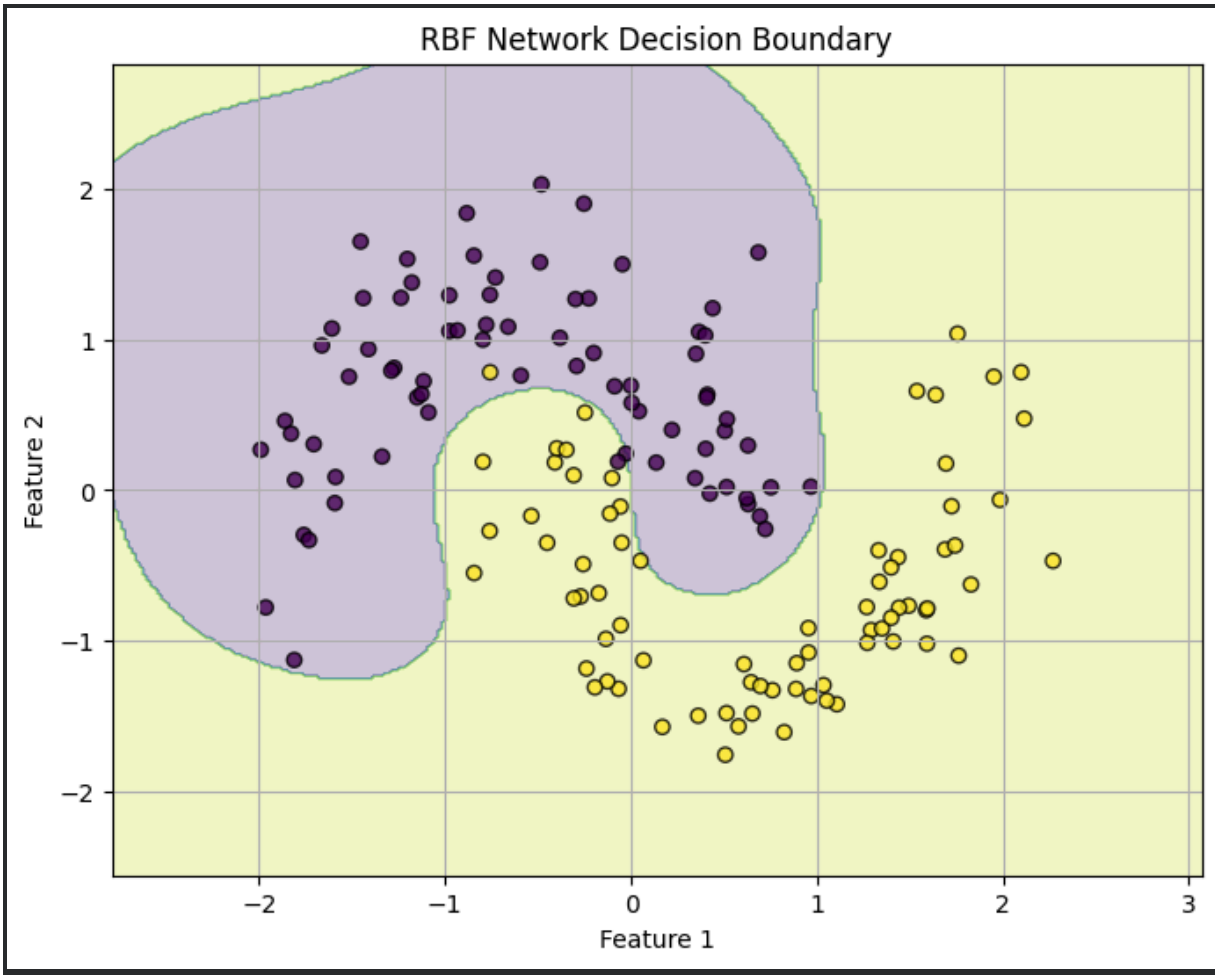
KNN Classification Report

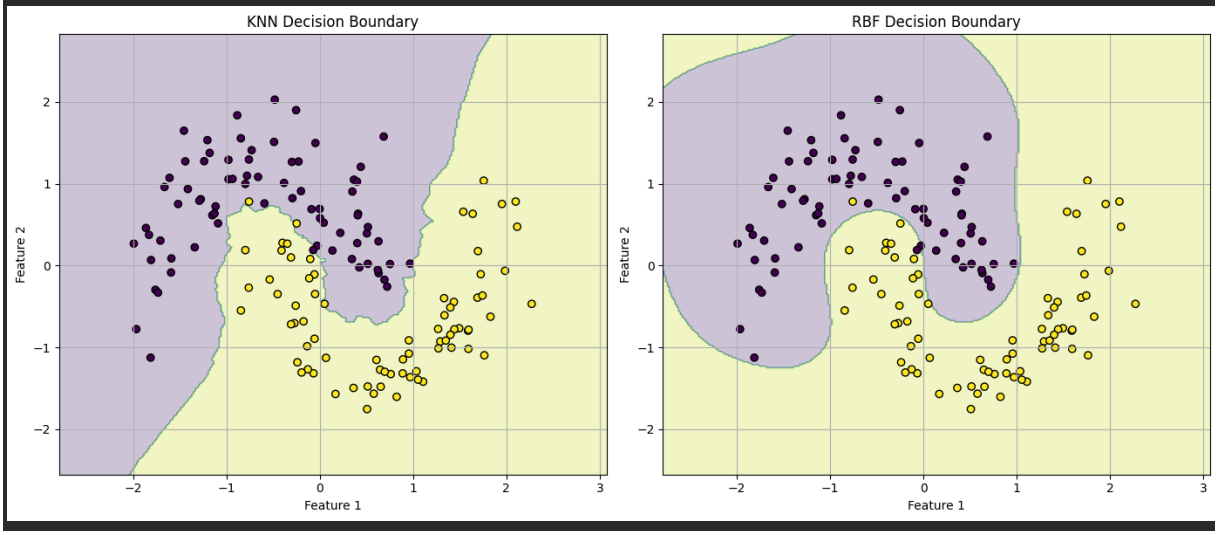
	precision	recall	f1-score	support
0	0.96	0.99	0.97	75
1	0.99	0.96	0.97	75
accuracy			0.97	150
macro avg	0.97	0.97	0.97	150
weighted avg	0.97	0.97	0.97	150

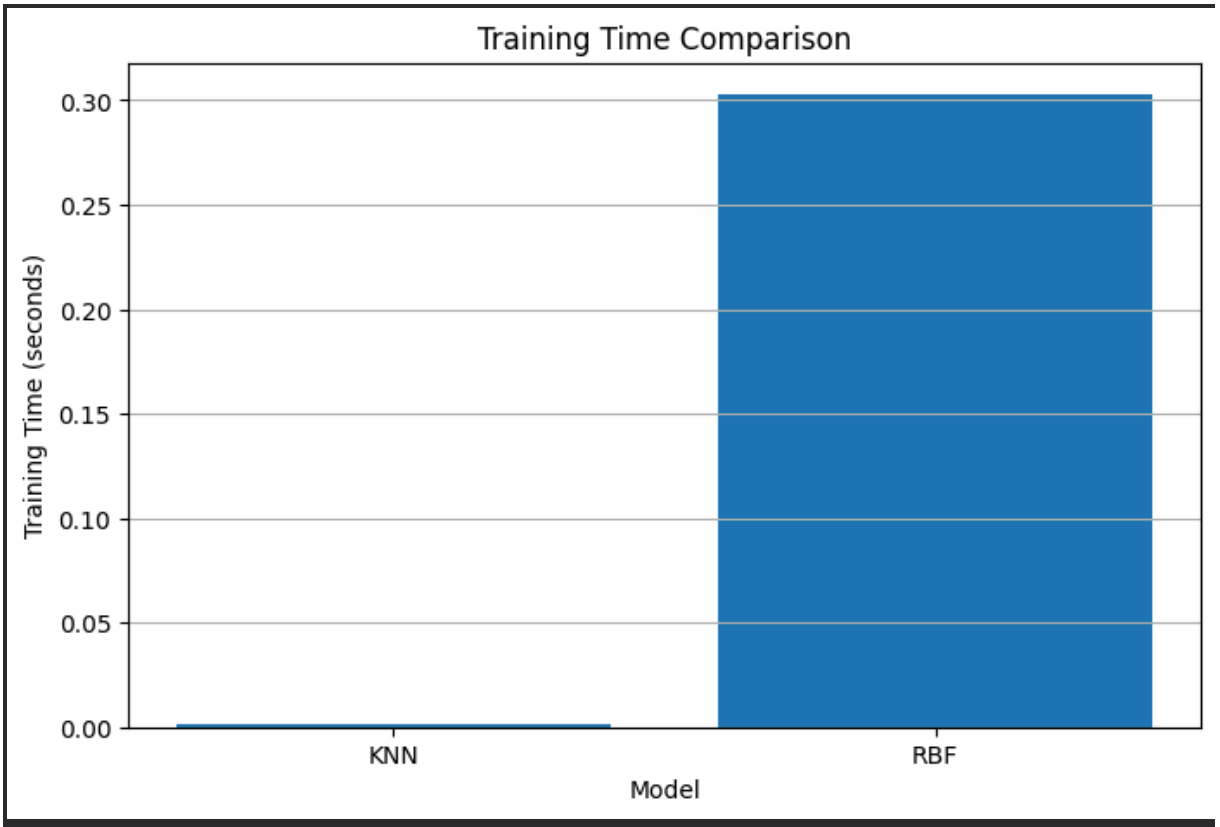
RBF Classification Report

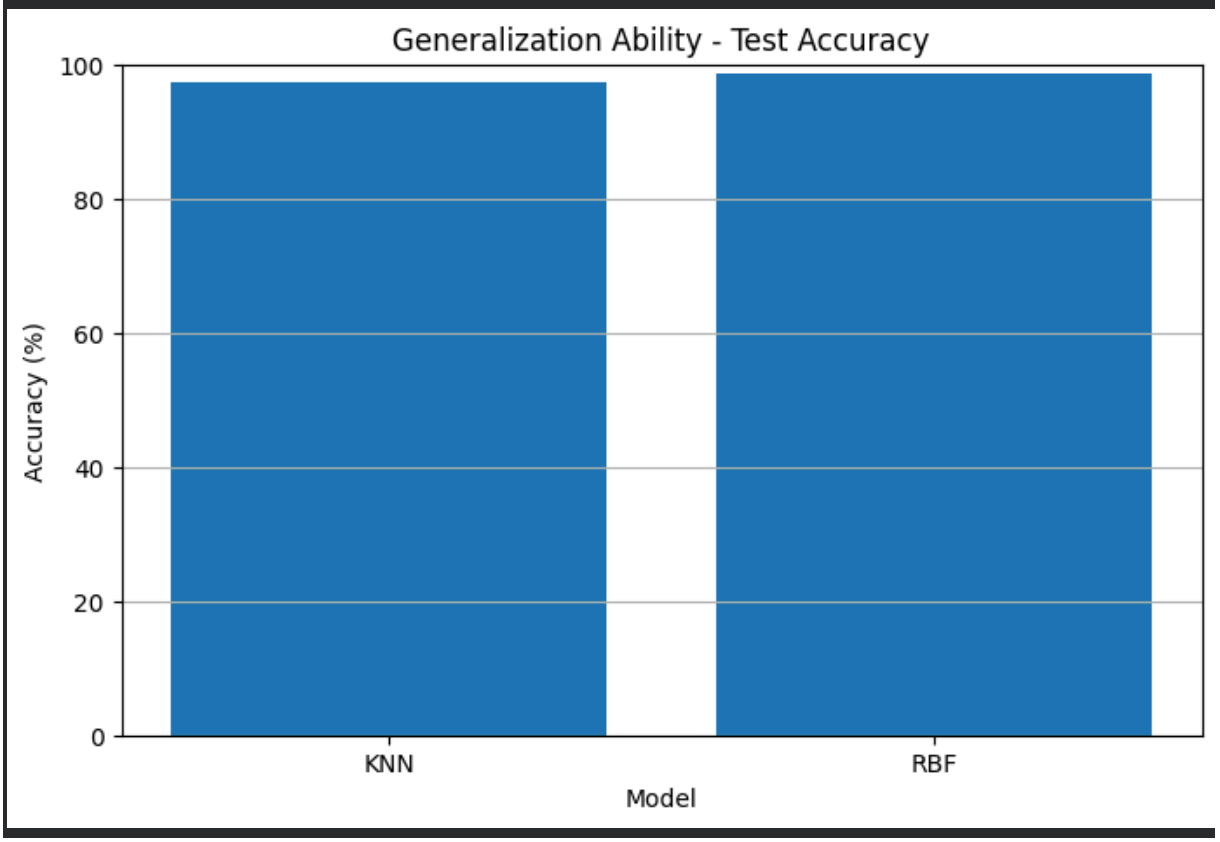
	precision	recall	f1-score	support
0	0.99	0.99	0.99	75
1	0.99	0.99	0.99	75
accuracy			0.99	150
macro avg	0.99	0.99	0.99	150
weighted avg	0.99	0.99	0.99	150


KNN Decision Boundary
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2
3


RBF Network Decision Boundary
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2
3


KNN Decision Boundary
RBF Decision Boundary
Feature 2
Feature 1


Training Time Comparison
Training Time (seconds)
0.30
0.25
0.20
0.15
0.10
0.05
0.00
KNN
RBF
Model


Generalization Ability - Test Accuracy
Accuracy (%)
100
80
60
40
20
0
KNN
RBF
Model

4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.

```
# ============================================================
# CASE-BASED LEARNING (CBL) vs LOCALLY WEIGHTED REGRESSION
# Practical Example: Movie Recommendation System
# Single Google Colab Code
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.metrics.pairwise import euclidean_distances
from sklearn.metrics import mean_squared_error

np.random.seed(42)

# ------------------------------------------------------------
# 1. CREATE PRACTICAL MOVIE RATING DATASET
# ------------------------------------------------------------

movies = [
    "Inception",
    "Interstellar",
    "Avatar",
    "Titanic",
    "The_Martian"
]

# Existing users' ratings
data = {
    "User": ["User1", "User2", "User3", "User4", "User5",
             "User6", "User7", "User8", "User9", "User10"],

    "Inception": [5, 5, 4, 2, 5, 4, 1, 5, 4, 2],
    "Interstellar": [5, 4, 5, 2, 5, 4, 1, 5, 4, 2],
    "Avatar": [4, 5, 4, 3, 4, 5, 2, 4, 5, 3],
    "Titanic": [2, 1, 2, 5, 2, 3, 5, 1, 2, 5],

    # Target movie
    "The_Martian": [5, 5, 4, 2, 5, 4, 1, 5, 4, 2]
```

```
}

df = pd.DataFrame(data)

print("=" * 70)
print("CASE-BASED LEARNING vs LOCALLY WEIGHTED REGRESSION")
print("Movie Recommendation System")
print("=" * 70)

print("\nDataset:")
print(df)



# ------------------------------------------------------------
# 2. NEW USER
# ------------------------------------------------------------

new_user = np.array([
    5,   # Inception
    5,   # Interstellar
    4,   # Avatar
    2    # Titanic
])

print("\nNew User Ratings:")
for i, movie in enumerate(movies[:4]):
    print(movie, ":", new_user[i])



# ------------------------------------------------------------
# 3. CASE-BASED LEARNING (CBL)
# ------------------------------------------------------------

start_cbl = time.time()

# Features = ratings of known movies
X_cases = df[
    ["Inception", "Interstellar", "Avatar", "Titanic"]
].values

# Target = rating of The Martian
y_cases = df["The_Martian"].values
```

```
# Calculate distance between new user and previous users
distances = np.linalg.norm(
    X_cases - new_user,
    axis=1
)

# Find most similar cases
k = 3

nearest_indices = np.argsort(distances)[:k]

nearest_users = df.iloc[nearest_indices]

# Average rating from similar users
cbl_prediction = np.mean(
    y_cases[nearest_indices]
)

cbl_time = time.time() - start_cbl

print("\n" + "-" * 70)
print("CASE-BASED LEARNING")
print("-" * 70)

print("\nMost Similar Users:")

for index in nearest_indices:
    print(
        df.iloc[index]["User"],
        "Distance:",
        round(distances[index], 3),
        "The Martian Rating:",
        df.iloc[index]["The_Martian"]
    )

print(
    "\nPredicted Rating for The Martian:",
    round(cbl_prediction, 2)
)

print(
    "Computation Time:",
    round(cbl_time, 6),
```

```
    "seconds"
)


# ------------------------------------------------------------
# 4. LOCALLY WEIGHTED REGRESSION (LWR)
# ------------------------------------------------------------

start_lwr = time.time()

# Gaussian bandwidth
bandwidth = 2.0

# Calculate distance
distances = np.linalg.norm(
    X_cases - new_user,
    axis=1
)

# Calculate Gaussian weights
weights = np.exp(
    -(distances ** 2) /
    (2 * bandwidth ** 2)
)

# Add small value to avoid division by zero
weights = weights + 1e-8

# Weighted prediction
lwr_prediction = np.sum(
    weights * y_cases
) / np.sum(weights)

lwr_time = time.time() - start_lwr

print("\n" + "-" * 70)
print("LOCALLY WEIGHTED REGRESSION")
print("-" * 70)

print("\nUser Similarity Weights:")

for i in range(len(df)):
    print(
```

```
        df.iloc[i]["User"],
        "Distance:",
        round(distances[i], 3),
        "Weight:",
        round(weights[i], 4)
    )

print(
    "\nPredicted Rating for The Martian:",
    round(lwr_prediction, 2)
)

print(
    "Computation Time:",
    round(lwr_time, 6),
    "seconds"
)



# ------------------------------------------------------------
# 5. COMPARE PREDICTIONS
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("PREDICTION COMPARISON")
print("=" * 70)

print(
    "\nCase-Based Learning Prediction:",
    round(cbl_prediction, 2)
)

print(
    "Locally Weighted Regression Prediction:",
    round(lwr_prediction, 2)
)



# ------------------------------------------------------------
# 6. VISUALIZE SIMILARITY
# ------------------------------------------------------------

plt.figure(figsize=(10, 5))
```

```
plt.bar(
    df["User"],
    distances
)

plt.xlabel("Users")
plt.ylabel("Distance from New User")
plt.title("Case Similarity - Distance from New User")
plt.xticks(rotation=45)
plt.grid(axis="y")

plt.show()


# ------------------------------------------------------------
# 7. VISUALIZE LWR WEIGHTS
# ------------------------------------------------------------

plt.figure(figsize=(10, 5))

plt.bar(
    df["User"],
    weights
)

plt.xlabel("Users")
plt.ylabel("LWR Weight")
plt.title("Locally Weighted Regression - User Weights")
plt.xticks(rotation=45)
plt.grid(axis="y")

plt.show()


# ------------------------------------------------------------
# 8. PREDICTION COMPARISON GRAPH
# ------------------------------------------------------------

models = [
    "Case-Based Learning",
    "Locally Weighted Regression"
]
```

```
predictions = [
    cbl_prediction,
    lwr_prediction
]

plt.figure(figsize=(8, 5))

plt.bar(
    models,
    predictions
)

plt.ylabel("Predicted Rating")
plt.title("Movie Rating Prediction Comparison")
plt.ylim(0, 5.5)
plt.grid(axis="y")

plt.show()



# ------------------------------------------------------------
# 9. COMPUTATIONAL EFFICIENCY
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("COMPUTATIONAL EFFICIENCY")
print("=" * 70)

print(
    "\nCBL Computation Time:",
    round(cbl_time, 6),
    "seconds"
)

print(
    "LWR Computation Time:",
    round(lwr_time, 6),
    "seconds"
)



# ------------------------------------------------------------
```

```
# 10. ADAPTABILITY TEST
# ------------------------------------------------------------

# Add a new user/case
new_case = np.array([
    5, 5, 5, 1
])

new_case_martian_rating = 5

X_cases_updated = np.vstack([
    X_cases,
    new_case
])

y_cases_updated = np.append(
    y_cases,
    new_case_martian_rating
)

# Recalculate distances
new_distances = np.linalg.norm(
    X_cases_updated - new_user,
    axis=1
)

nearest_indices_updated = np.argsort(
    new_distances
)[:k]

updated_cbl_prediction = np.mean(
    y_cases_updated[nearest_indices_updated]
)

print("\n" + "=" * 70)
print("ADAPTABILITY TEST")
print("=" * 70)

print("\nNew Case Added Successfully.")

print(
    "Updated CBL Prediction:",
    round(updated_cbl_prediction, 2)
```

```
)

print("""
CBL:
New cases can be directly added to the case database
without retraining a complete model.

LWR:
New observations can also be incorporated immediately,
but similarity weights and local calculations are
performed again for each new prediction.
""")



# ------------------------------------------------------------
# 11. FINAL ANALYSIS
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("FINAL ANALYSIS")
print("=" * 70)
```

```
======================================================================
CASE-BASED LEARNING vs LOCALLY WEIGHTED REGRESSION
Movie Recommendation System
======================================================================

Dataset:
    User  Inception  Interstellar  Avatar  Titanic  The_Martian
0   User1          5             5       4        2            5
1   User2          5             4       5        1            5
2   User3          4             5       4        2            4
3   User4          2             2       3        5            2
4   User5          5             5       4        2            5
5   User6          4             4       5        3            4
6   User7          1             1       2        5            1
7   User8          5             5       4        1            5
8   User9          4             4       5        2            4
9  User10          2             2       3        5            2

New User Ratings:
Inception : 5
Interstellar : 5
Avatar : 4
Titanic : 2

----------------------------------------------------------------------
CASE-BASED LEARNING
----------------------------------------------------------------------
```

Most Similar Users:
User1 Distance: 0.0 The Martian Rating: 5
User5 Distance: 0.0 The Martian Rating: 5
User8 Distance: 1.0 The Martian Rating: 5

Predicted Rating for The Martian: 5.0
Computation Time: 0.012095 seconds

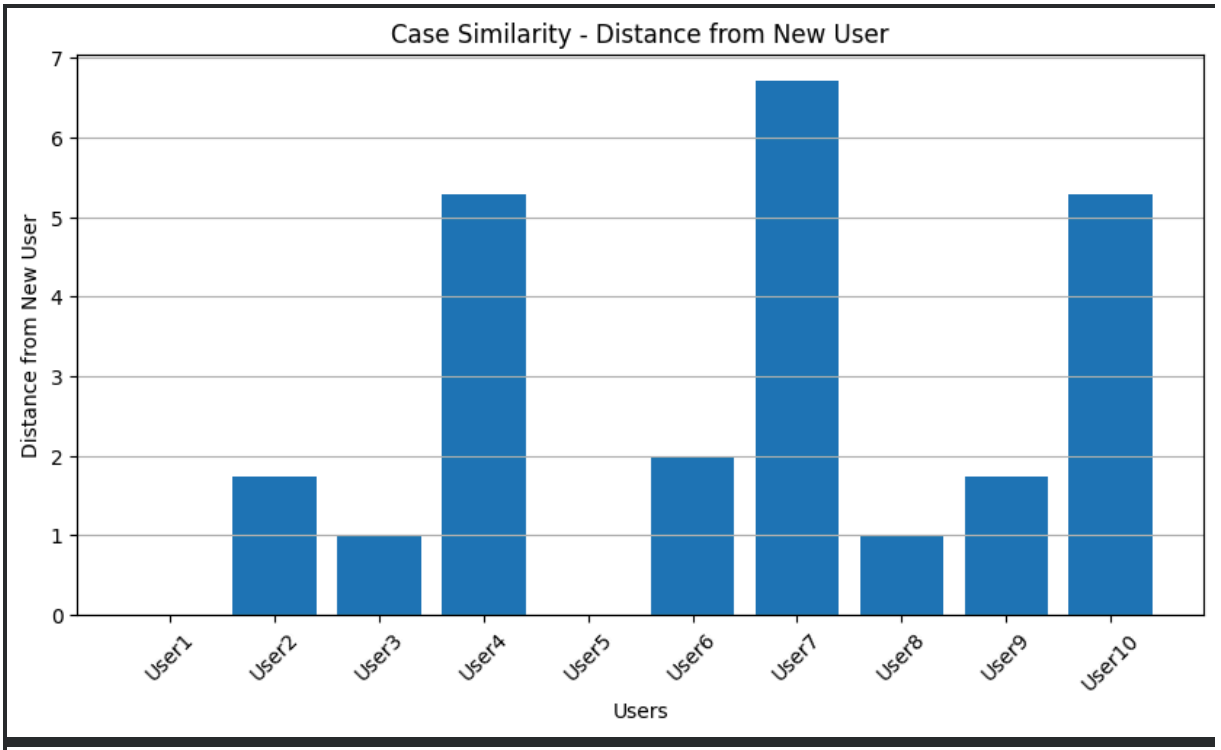
LOCALLY WEIGHTED REGRESSION

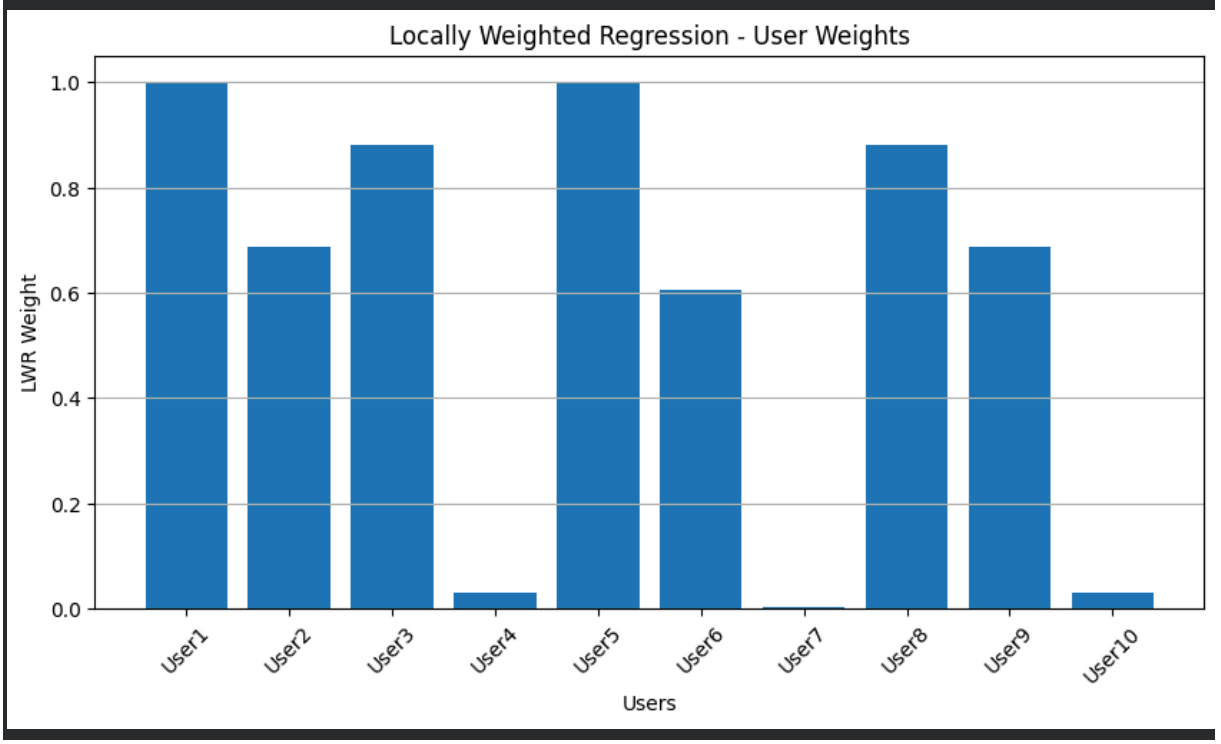
User Similarity Weights:
User1 Distance: 0.0 Weight: 1.0
User2 Distance: 1.732 Weight: 0.6873
User3 Distance: 1.0 Weight: 0.8825
User4 Distance: 5.292 Weight: 0.0302
User5 Distance: 0.0 Weight: 1.0
User6 Distance: 2.0 Weight: 0.6065
User7 Distance: 6.708 Weight: 0.0036
User8 Distance: 1.0 Weight: 0.8825
User9 Distance: 1.732 Weight: 0.6873
User10 Distance: 5.292 Weight: 0.0302

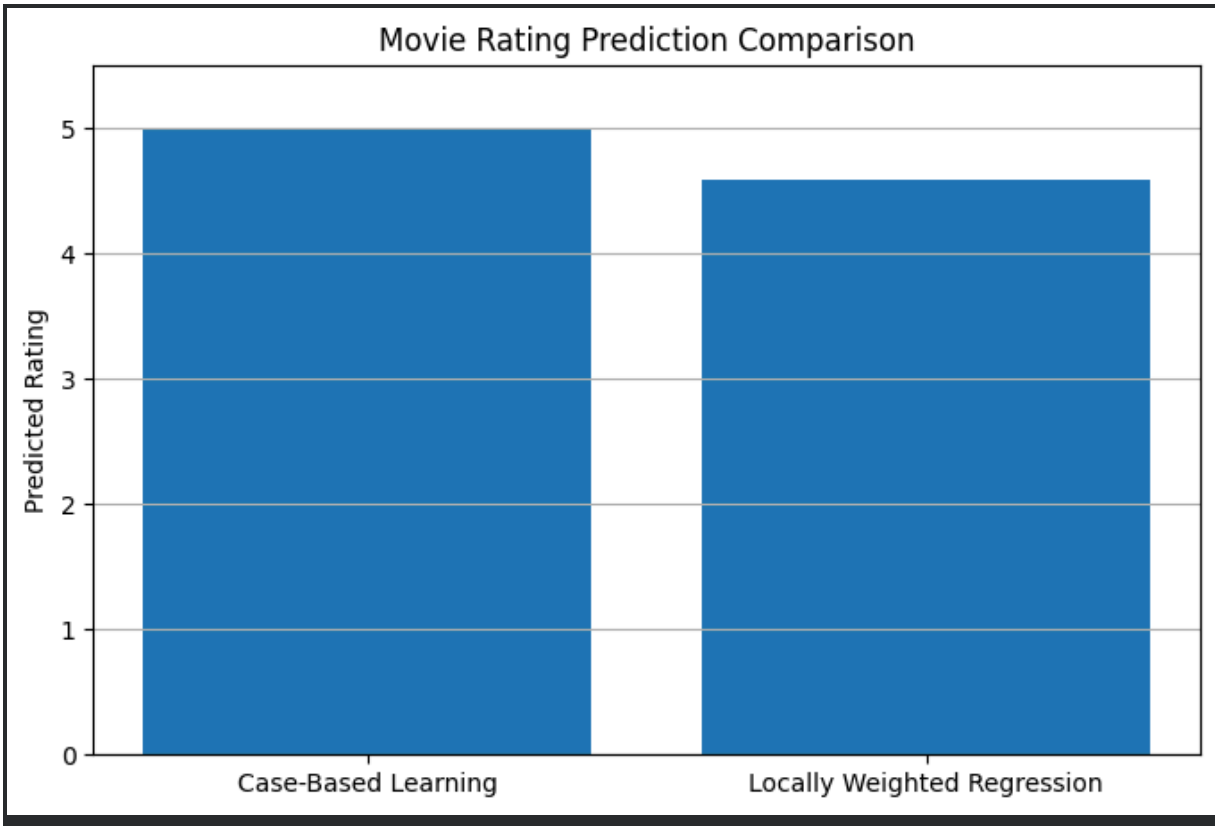
Predicted Rating for The Martian: 4.59
Computation Time: 0.000366 seconds

==========
PREDICTION COMPARISON
==========

Case-Based Learning Prediction: 5.0
Locally Weighted Regression Prediction: 4.59


Case Similarity - Distance from New User
Distance from New User
0
1
2
3
4
5
6
7
User1
User2
User3
User4
User5
User6
User7
User8
User9
User10
Users


Locally Weighted Regression - User Weights
LWR Weight
0.0
0.2
0.4
0.6
0.8
1.0
User1
User2
User3
User4
User5
User6
User7
User8
User9
User10
Users




```
==========================================================================
COMPUTATIONAL EFFICIENCY
==========================================================================

CBL Computation Time: 0.012095 seconds
LWR Computation Time: 0.000366 seconds

==========================================================================
ADAPTABILITY TEST
==========================================================================

New Case Added Successfully.
Updated CBL Prediction: 5.0
```

6. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

```
# ============================================================
# NAIVE BAYES vs LOGISTIC REGRESSION
# Text Classification - Spam Detection
# Single Google Colab Code
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import time

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report
)

# ------------------------------------------------------------
# 1. CREATE SAMPLE SPAM DATASET
# ------------------------------------------------------------

messages = [
    "Congratulations you won a free lottery prize",
    "Win a free iPhone now click here",
    "You have won a cash prize claim now",
    "Free money waiting for you click now",
    "Congratulations you are selected for a free gift",
    "Claim your free reward today",
    "Win cash instantly by clicking this link",
    "Exclusive offer free vacation for you",
    "You won a free shopping voucher",
    "Get free money now limited offer",
    "Claim your lottery prize immediately",
    "Free gift card waiting for you",
    "Congratulations winner claim your reward",
    "Win a brand new phone for free",
    "Urgent claim your cash prize now",
    "Free bonus available click here",
    "You are a lucky winner receive cash",
    "Special offer win free products",
    "Get your free coupon today",
    "Congratulations free prize waiting",

    "Can we meet tomorrow for the project",
    "Please send me the assignment details",
```

```
    "Your meeting is scheduled at 10 AM",
    "Please call me when you are available",
    "The project report is ready",
    "Can you send the notes from class",
    "Your interview is confirmed for tomorrow",
    "Please submit the document before Friday",
    "The team meeting starts at 2 PM",
    "Let us discuss the project today",
    "I will send the files by evening",
    "Please check your email for the report",
    "The class starts at 9 AM tomorrow",
    "Can you help me with this assignment",
    "The presentation is scheduled next week",
    "Please attend the meeting tomorrow",
    "Your order has been delivered successfully",
    "Thank you for your help",
    "See you at the office tomorrow",
    "Please review the project document"
]

labels = [
    "spam", "spam", "spam", "spam", "spam",
    "spam", "spam", "spam", "spam", "spam",
    "spam", "spam", "spam", "spam", "spam",
    "spam", "spam", "spam", "spam", "spam",

    "ham", "ham", "ham", "ham", "ham",
    "ham", "ham", "ham", "ham", "ham",
    "ham", "ham", "ham", "ham", "ham",
    "ham", "ham", "ham", "ham", "ham"
]

df = pd.DataFrame({
    "message": messages,
    "label": labels
})

print("=" * 70)
print("NAIVE BAYES vs LOGISTIC REGRESSION - SPAM DETECTION")
print("=" * 70)

print("\nDataset:")
print(df.head())
```

```
print("\nClass Distribution:")
print(df["label"].value_counts())


# ------------------------------------------------------------
# 2. SPLIT DATASET
# ------------------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    df["message"],
    df["label"],
    test_size=0.30,
    random_state=42,
    stratify=df["label"]
)


# ------------------------------------------------------------
# 3. TF-IDF FEATURE EXTRACTION
# ------------------------------------------------------------

vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    ngram_range=(1, 2)
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print("\nTF-IDF Feature Matrix:")
print("Training Shape:", X_train_tfidf.shape)
print("Testing Shape:", X_test_tfidf.shape)


# ------------------------------------------------------------
# 4. NAIVE BAYES
# ------------------------------------------------------------

start_nb = time.time()

nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
```

```
nb_training_time = time.time() - start_nb

start_nb_prediction = time.time()

y_pred_nb = nb_model.predict(X_test_tfidf)

nb_prediction_time = time.time() - start_nb_prediction


# ------------------------------------------------------------
# 5. LOGISTIC REGRESSION
# ------------------------------------------------------------

start_lr = time.time()

lr_model = LogisticRegression(
    max_iter=1000
)

lr_model.fit(X_train_tfidf, y_train)

lr_training_time = time.time() - start_lr

start_lr_prediction = time.time()

y_pred_lr = lr_model.predict(X_test_tfidf)

lr_prediction_time = time.time() - start_lr_prediction


# ------------------------------------------------------------
# 6. PERFORMANCE METRICS
# ------------------------------------------------------------

nb_accuracy = accuracy_score(y_test, y_pred_nb)
nb_precision = precision_score(
    y_test,
    y_pred_nb,
    pos_label="spam"
)
nb_recall = recall_score(
    y_test,
    y_pred_nb,
    pos_label="spam"
```

```
)
nb_f1 = f1_score(
    y_test,
    y_pred_nb,
    pos_label="spam"
)

lr_accuracy = accuracy_score(y_test, y_pred_lr)
lr_precision = precision_score(
    y_test,
    y_pred_lr,
    pos_label="spam"
)
lr_recall = recall_score(
    y_test,
    y_pred_lr,
    pos_label="spam"
)
lr_f1 = f1_score(
    y_test,
    y_pred_lr,
    pos_label="spam"
)


# ------------------------------------------------------------
# 7. PRINT RESULTS
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("PERFORMANCE COMPARISON")
print("=" * 70)

results = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score",
        "Training Time (sec)",
        "Prediction Time (sec)"
    ],

    "Naive Bayes": [
```

```
        nb_accuracy,
        nb_precision,
        nb_recall,
        nb_f1,
        nb_training_time,
        nb_prediction_time
    ],

    "Logistic Regression": [
        lr_accuracy,
        lr_precision,
        lr_recall,
        lr_f1,
        lr_training_time,
        lr_prediction_time
    ]
})

print(results.to_string(index=False))


# ------------------------------------------------------------
# 8. CLASSIFICATION REPORT
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("NAIVE BAYES CLASSIFICATION REPORT")
print("=" * 70)

print(
    classification_report(
        y_test,
        y_pred_nb
    )
)

print("\n" + "=" * 70)
print("LOGISTIC REGRESSION CLASSIFICATION REPORT")
print("=" * 70)

print(
    classification_report(
        y_test,
        y_pred_lr
```

```
    )
)


# ----------------------------------------------------------
# 9. CONFUSION MATRICES
# ----------------------------------------------------------

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

cm_nb = confusion_matrix(
    y_test,
    y_pred_nb,
    labels=["ham", "spam"]
)

cm_lr = confusion_matrix(
    y_test,
    y_pred_lr,
    labels=["ham", "spam"]
)

ConfusionMatrixDisplay(
    cm_nb,
    display_labels=["Ham", "Spam"]
).plot(
    ax=axes[0],
    colorbar=False
)

axes[0].set_title("Naive Bayes Confusion Matrix")

ConfusionMatrixDisplay(
    cm_lr,
    display_labels=["Ham", "Spam"]
).plot(
    ax=axes[1],
    colorbar=False
)

axes[1].set_title("Logistic Regression Confusion Matrix")

plt.tight_layout()
plt.show()
```

```
# ------------------------------------------------------------
# 10. ACCURACY COMPARISON
# ------------------------------------------------------------

models = [
    "Naive Bayes",
    "Logistic Regression"
]

accuracies = [
    nb_accuracy * 100,
    lr_accuracy * 100
]

plt.figure(figsize=(8, 5))

plt.bar(
    models,
    accuracies
)

plt.ylabel("Accuracy (%)")
plt.xlabel("Model")
plt.title("Naive Bayes vs Logistic Regression Accuracy")
plt.ylim(0, 105)
plt.grid(axis="y")

plt.show()


# ------------------------------------------------------------
# 11. TRAINING TIME COMPARISON
# ------------------------------------------------------------

training_times = [
    nb_training_time,
    lr_training_time
]

plt.figure(figsize=(8, 5))

plt.bar(
```

```
    models,
    training_times
)

plt.ylabel("Training Time (seconds)")
plt.xlabel("Model")
plt.title("Training Time Comparison")
plt.grid(axis="y")

plt.show()


# ------------------------------------------------------------
# 12. PRECISION, RECALL AND F1 COMPARISON
# ------------------------------------------------------------

metrics = [
    "Precision",
    "Recall",
    "F1 Score"
]

nb_values = [
    nb_precision,
    nb_recall,
    nb_f1
]

lr_values = [
    lr_precision,
    lr_recall,
    lr_f1
]

x = np.arange(len(metrics))
width = 0.35

plt.figure(figsize=(9, 5))

plt.bar(
    x - width / 2,
    nb_values,
    width,
    label="Naive Bayes"
```

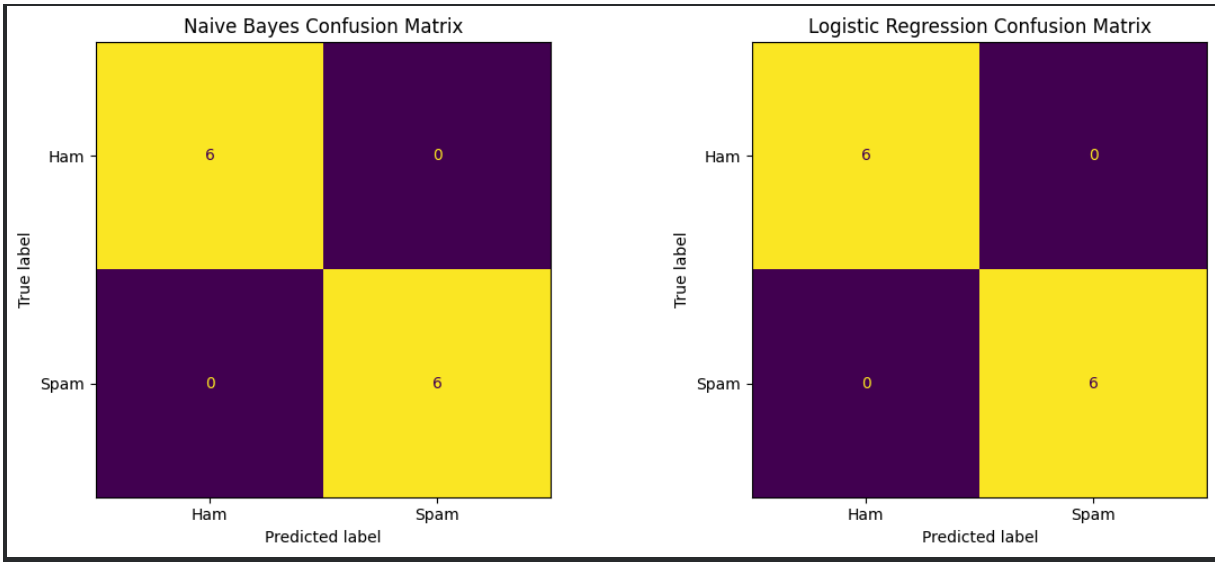
```
)

plt.bar(
    x + width / 2,
    lr_values,
    width,
    label="Logistic Regression"
)

plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Performance Metric Comparison")
plt.ylim(0, 1.1)
plt.legend()
plt.grid(axis="y")

plt.show()


# ------------------------------------------------------------
# 13. TEST NEW EMAILS
# ------------------------------------------------------------

new_emails = [
    "Congratulations you won a free cash prize",
    "Please send me the project report",
    "Win a free gift click now",
    "The meeting is scheduled tomorrow at 10 AM"
]

new_email_tfidf = vectorizer.transform(new_emails)

nb_new_predictions = nb_model.predict(
    new_email_tfidf
)

lr_new_predictions = lr_model.predict(
    new_email_tfidf
)

print("\n" + "=" * 70)
print("NEW EMAIL PREDICTIONS")
print("=" * 70)
```

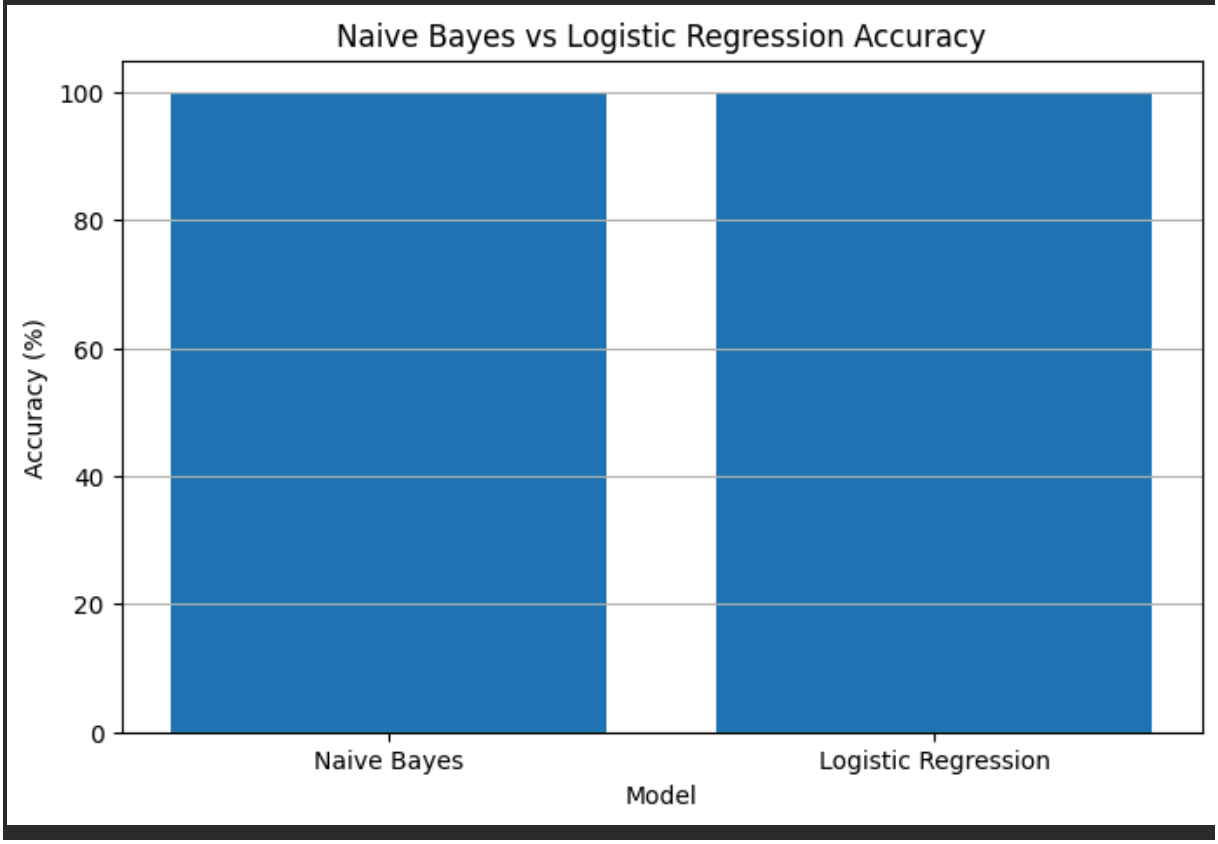
```
for i, email in enumerate(new_emails):

    print("\nEmail:", email)

    print(
        "Naive Bayes:",
        nb_new_predictions[i]
    )

    print(
        "Logistic Regression:",
        lr_new_predictions[i]
    )


# ------------------------------------------------------------
# 14. HIGH-DIMENSIONAL DATA TEST
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("HIGH-DIMENSIONAL TEXT DATA ANALYSIS")
print("=" * 70)

print(
    "\nNumber of TF-IDF Features:",
    X_train_tfidf.shape[1]
)

print("""
Naive Bayes:
- Very efficient for high-dimensional sparse text data.
- Uses word probability estimates.
- Assumes conditional independence between features.

Logistic Regression:
- Handles high-dimensional sparse data effectively.
- Does not require feature independence.
- Learns feature weights using optimization.
""")
```

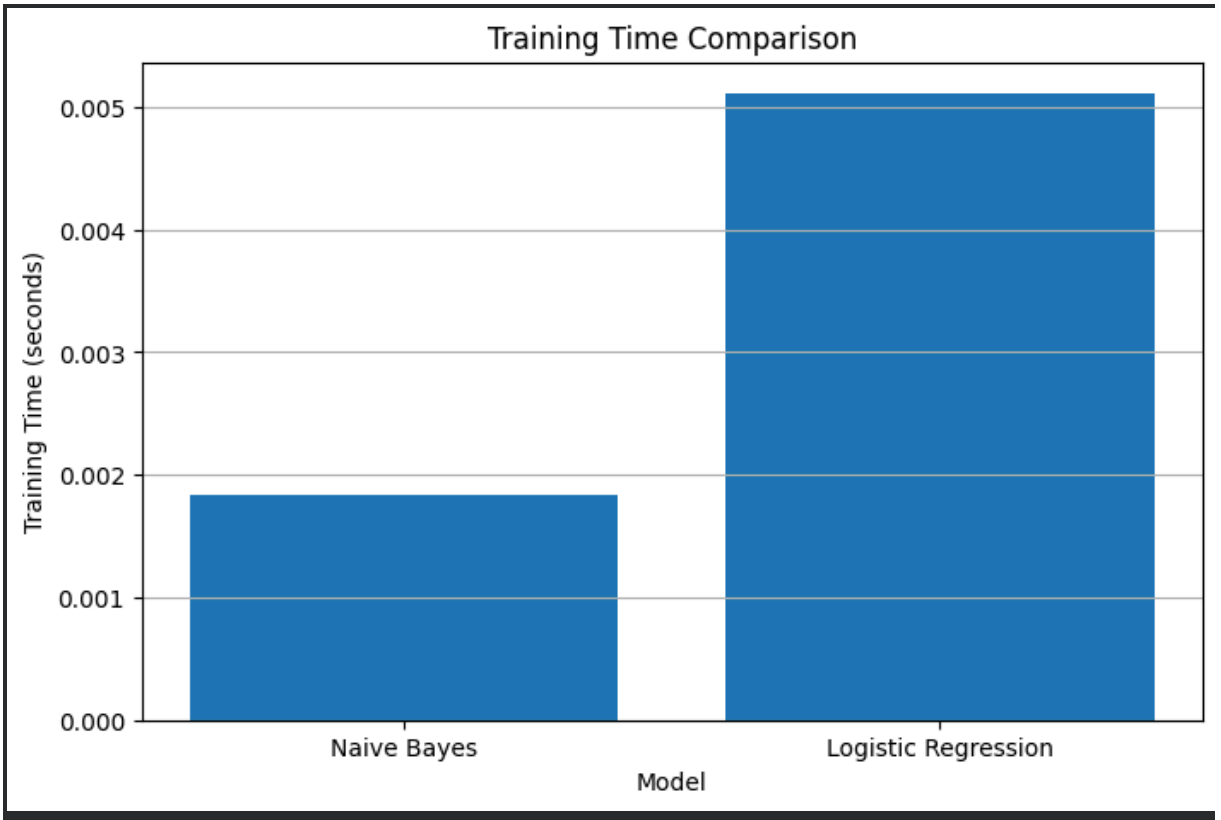
```
======================================================================
NAIVE BAYES vs LOGISTIC REGRESSION - SPAM DETECTION
======================================================================

Dataset:
```

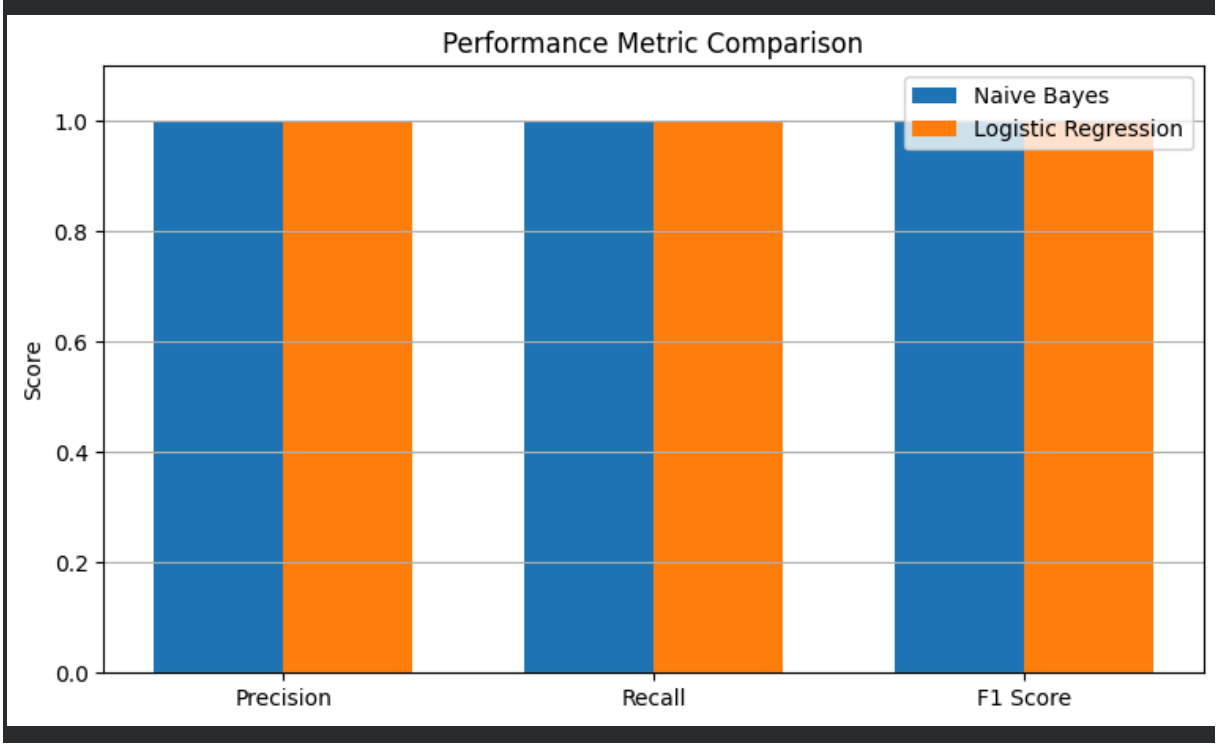
```
                                   message label
0      Congratulations you won a free lottery prize  spam
1                 Win a free iPhone now click here  spam
2               You have won a cash prize claim now  spam
3              Free money waiting for you click now  spam
4  Congratulations you are selected for a free gift  spam

Class Distribution:
label
spam    20
ham     20
Name: count, dtype: int64

TF-IDF Feature Matrix:
Training Shape: (28, 127)
Testing Shape: (12, 127)

======================================================================
PERFORMANCE COMPARISON
======================================================================
              Metric  Naive Bayes  Logistic Regression
            Accuracy     1.000000             1.000000
           Precision     1.000000             1.000000
              Recall     1.000000             1.000000
            F1 Score     1.000000             1.000000
  Training Time (sec)     0.001841             0.005110
Prediction Time (sec)     0.000388             0.000498

======================================================================
NAIVE BAYES CLASSIFICATION REPORT
======================================================================
              precision    recall  f1-score   support

         ham       1.00      1.00      1.00         6
        spam       1.00      1.00      1.00         6

    accuracy                           1.00        12
   macro avg       1.00      1.00      1.00        12
weighted avg       1.00      1.00      1.00        12


======================================================================
LOGISTIC REGRESSION CLASSIFICATION REPORT
======================================================================
              precision    recall  f1-score   support

         ham       1.00      1.00      1.00         6
        spam       1.00      1.00      1.00         6

    accuracy                           1.00        12
```

macro avg	1.00	1.00	1.00	12
weighted avg	1.00	1.00	1.00	12









10. Compare Principal Component Analysis (PCA) and Feature Selection techniques for dimensionality reduction. Analyze differences in information preservation,

computational complexity, interpretability, and impact on model performance. Discuss how each approach affects downstream machine learning tasks.

```
# ============================================================
# PCA vs FEATURE SELECTION
# Dimensionality Reduction Comparison
# Single Google Colab Code
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# ------------------------------------------------------------
# 1. LOAD DATASET
# ------------------------------------------------------------

data = load_breast_cancer()

X = data.data
y = data.target

feature_names = data.feature_names

print("=" * 70)
print("PCA vs FEATURE SELECTION")
print("Dimensionality Reduction for Machine Learning")
print("=" * 70)

print("\nOriginal Dataset Shape:")
print("Samples:", X.shape[0])
print("Features:", X.shape[1])



# ------------------------------------------------------------
# 2. TRAIN-TEST SPLIT
```

```
# ----------------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)


# ----------------------------------------------------------
# 3. STANDARDIZATION
# ----------------------------------------------------------

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# ============================================================
# 4. PCA
# ============================================================

start_pca = time.time()

pca = PCA(
    n_components=10
)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

pca_time = time.time() - start_pca

explained_variance = pca.explained_variance_ratio_
total_variance = np.sum(explained_variance)

print("\n" + "=" * 70)
print("PCA RESULTS")
print("=" * 70)

print("\nOriginal Features:", X.shape[1])
```

```
print("PCA Components:", X_train_pca.shape[1])

print(
    "Explained Variance:",
    round(total_variance * 100, 2),
    "%"
)

print(
    "PCA Computation Time:",
    round(pca_time, 6),
    "seconds"
)


# ============================================================
# 5. FEATURE SELECTION
# ============================================================

start_fs = time.time()

selector = SelectKBest(
    score_func=f_classif,
    k=10
)

X_train_fs = selector.fit_transform(
    X_train_scaled,
    y_train
)

X_test_fs = selector.transform(
    X_test_scaled
)

fs_time = time.time() - start_fs

selected_features = feature_names[
    selector.get_support()
]

print("\n" + "=" * 70)
print("FEATURE SELECTION RESULTS")
print("=" * 70)
```

```
print("\nOriginal Features:", X.shape[1])
print("Selected Features:", X_train_fs.shape[1])

print("\nSelected Features:")

for feature in selected_features:
    print("-", feature)

print(
    "\nFeature Selection Time:",
    round(fs_time, 6),
    "seconds"
)


# ============================================================
# 6. LOGISTIC REGRESSION WITH ORIGINAL FEATURES
# ============================================================

start_original = time.time()

model_original = LogisticRegression(
    max_iter=5000
)

model_original.fit(
    X_train_scaled,
    y_train
)

y_pred_original = model_original.predict(
    X_test_scaled
)

original_time = time.time() - start_original

original_accuracy = accuracy_score(
    y_test,
    y_pred_original
)


# ============================================================
```

```
# 7. LOGISTIC REGRESSION WITH PCA
# ============================================================

start_pca_model = time.time()

model_pca = LogisticRegression(
    max_iter=5000
)

model_pca.fit(
    X_train_pca,
    y_train
)

y_pred_pca = model_pca.predict(
    X_test_pca
)

pca_model_time = time.time() - start_pca_model

pca_accuracy = accuracy_score(
    y_test,
    y_pred_pca
)


# ============================================================
# 8. LOGISTIC REGRESSION WITH FEATURE SELECTION
# ============================================================

start_fs_model = time.time()

model_fs = LogisticRegression(
    max_iter=5000
)

model_fs.fit(
    X_train_fs,
    y_train
)

y_pred_fs = model_fs.predict(
    X_test_fs
)
```

```
fs_model_time = time.time() - start_fs_model

fs_accuracy = accuracy_score(
    y_test,
    y_pred_fs
)


# ============================================================
# 9. ACCURACY COMPARISON
# ============================================================

print("\n" + "=" * 70)
print("MODEL PERFORMANCE COMPARISON")
print("=" * 70)

print(
    "\nOriginal Features Accuracy:",
    round(original_accuracy * 100, 2),
    "%"
)

print(
    "PCA Accuracy:",
    round(pca_accuracy * 100, 2),
    "%"
)

print(
    "Feature Selection Accuracy:",
    round(fs_accuracy * 100, 2),
    "%"
)


# ============================================================
# 10. PERFORMANCE TABLE
# ============================================================

results = pd.DataFrame({
    "Method": [
        "Original Features",
        "PCA",
```

```
        "Feature Selection"
    ],

    "Number of Features": [
        X.shape[1],
        X_train_pca.shape[1],
        X_train_fs.shape[1]
    ],

    "Accuracy (%)": [
        original_accuracy * 100,
        pca_accuracy * 100,
        fs_accuracy * 100
    ],

    "Reduction Time (sec)": [
        0,
        pca_time,
        fs_time
    ]
})

print("\nPerformance Table:")
print(results.round(4).to_string(index=False))


# ============================================================
# 11. PCA EXPLAINED VARIANCE
# ============================================================

plt.figure(figsize=(10, 5))

components = np.arange(
    1,
    len(pca.explained_variance_ratio_) + 1
)

plt.bar(
    components,
    pca.explained_variance_ratio_ * 100
)

plt.xlabel("Principal Component")
plt.ylabel("Explained Variance (%)")
```

```
plt.title("PCA Explained Variance")
plt.grid(axis="y")

plt.show()


# ============================================================
# 12. CUMULATIVE PCA VARIANCE
# ============================================================

full_pca = PCA()

full_pca.fit(X_train_scaled)

cumulative_variance = np.cumsum(
    full_pca.explained_variance_ratio_
)

plt.figure(figsize=(10, 5))

plt.plot(
    range(1, len(cumulative_variance) + 1),
    cumulative_variance * 100,
    marker="o"
)

plt.axhline(
    90,
    linestyle="--",
    label="90% Variance"
)

plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance (%)")
plt.title("PCA Cumulative Explained Variance")
plt.legend()
plt.grid(True)

plt.show()


# ============================================================
# 13. PCA 2D VISUALIZATION
# ============================================================
```

```
pca_2d = PCA(
    n_components=2
)

X_pca_2d = pca_2d.fit_transform(
    X_train_scaled
)

plt.figure(figsize=(9, 6))

plt.scatter(
    X_pca_2d[:, 0],
    X_pca_2d[:, 1],
    c=y_train,
    alpha=0.7
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Data Representation Using PCA")
plt.grid(True)

plt.show()


# ============================================================
# 14. FEATURE IMPORTANCE / F-SCORE
# ============================================================

scores = selector.scores_

feature_scores = pd.DataFrame({
    "Feature": feature_names,
    "F-Score": scores
})

feature_scores = feature_scores.sort_values(
    by="F-Score",
    ascending=False
)

print("\n" + "=" * 70)
print("FEATURE SELECTION RANKING")
```

```
print("=" * 70)

print(
    feature_scores.head(10).round(3).to_string(index=False)
)


# ============================================================
# 15. FEATURE SELECTION VISUALIZATION
# ============================================================

top_features = feature_scores.head(10)

plt.figure(figsize=(10, 6))

plt.barh(
    top_features["Feature"][::-1],
    top_features["F-Score"][::-1]
)

plt.xlabel("F-Score")
plt.ylabel("Feature")
plt.title("Top Features Selected by ANOVA F-Test")
plt.grid(axis="x")

plt.show()


# ============================================================
# 16. MODEL ACCURACY GRAPH
# ============================================================

methods = [
    "Original",
    "PCA",
    "Feature Selection"
]

accuracies = [
    original_accuracy * 100,
    pca_accuracy * 100,
    fs_accuracy * 100
]
```

```
plt.figure(figsize=(9, 5))

plt.bar(
    methods,
    accuracies
)

plt.ylabel("Accuracy (%)")
plt.xlabel("Dimensionality Reduction Method")
plt.title("Downstream Classification Performance")
plt.ylim(0, 100)
plt.grid(axis="y")

plt.show()


# ============================================================
# 17. NUMBER OF FEATURES COMPARISON
# ============================================================

feature_counts = [
    X.shape[1],
    X_train_pca.shape[1],
    X_train_fs.shape[1]
]

plt.figure(figsize=(9, 5))

plt.bar(
    methods,
    feature_counts
)

plt.ylabel("Number of Features")
plt.xlabel("Method")
plt.title("Dimensionality Comparison")
plt.grid(axis="y")

plt.show()


# ============================================================
# 18. CLASSIFICATION REPORTS
# ============================================================
```

```
print("\n" + "=" * 70)
print("ORIGINAL FEATURES - CLASSIFICATION REPORT")
print("=" * 70)

print(
    classification_report(
        y_test,
        y_pred_original
    )
)

print("\n" + "=" * 70)
print("PCA - CLASSIFICATION REPORT")
print("=" * 70)

print(
    classification_report(
        y_test,
        y_pred_pca
    )
)

print("\n" + "=" * 70)
print("FEATURE SELECTION - CLASSIFICATION REPORT")
print("=" * 70)

print(
    classification_report(
        y_test,
        y_pred_fs
    )
)
```

```
======================================================================
PCA vs FEATURE SELECTION
Dimensionality Reduction for Machine Learning
======================================================================

Original Dataset Shape:
Samples: 569
Features: 30

======================================================================
PCA RESULTS
======================================================================
```

Original Features: 30
PCA Components: 10
Explained Variance: 95.62 %
PCA Computation Time: 0.003955 seconds

==========
FEATURE SELECTION RESULTS
==========

Original Features: 30
Selected Features: 10

Selected Features:
- mean radius
- mean perimeter
- mean area
- mean concavity
- mean concave points
- worst radius
- worst perimeter
- worst area
- worst concavity
- worst concave points

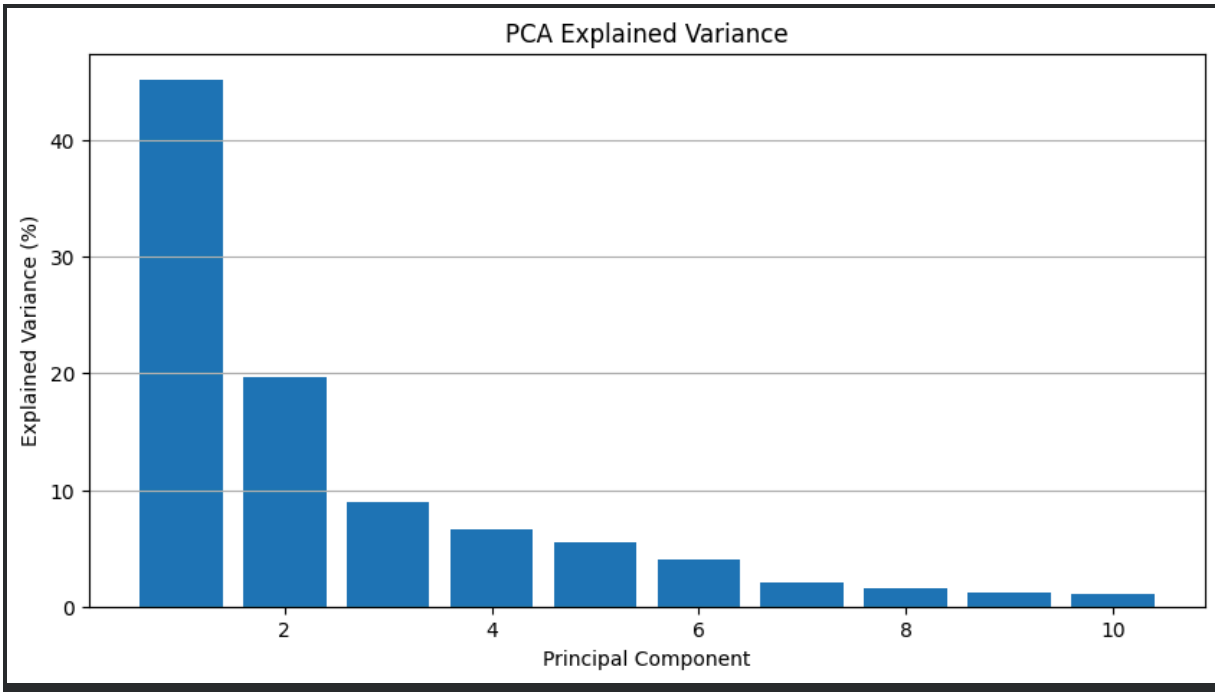
Feature Selection Time: 0.006159 seconds

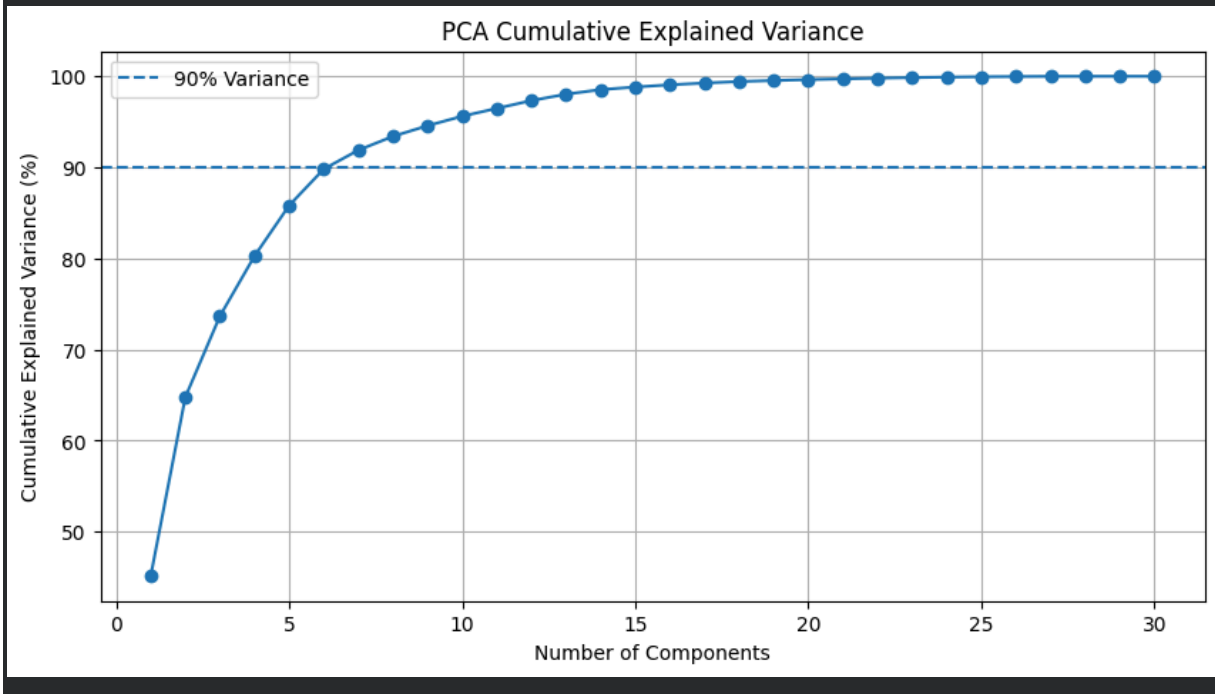
==========
MODEL PERFORMANCE COMPARISON
==========

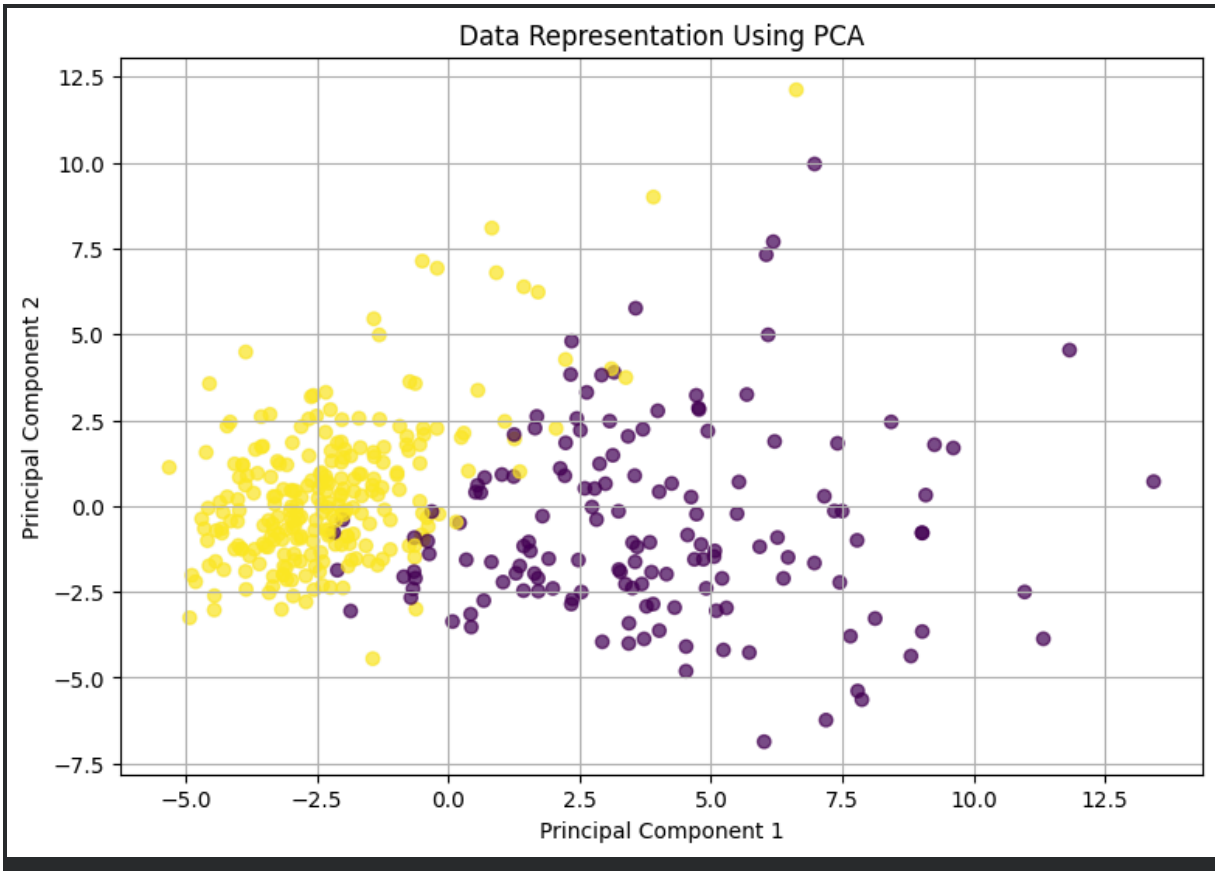
Original Features Accuracy: 98.83 %
PCA Accuracy: 97.08 %
Feature Selection Accuracy: 93.57 %

Performance Table:

Method	Number of Features	Accuracy (%)	Reduction Time (sec)
Original Features	30	98.8304	0.0000
PCA	10	97.0760	0.0040
Feature Selection	10	93.5673	0.0062


PCA Explained Variance
Explained Variance (%)
0
10
20
30
40
2
4
6
8
10
Principal Component


PCA Cumulative Explained Variance
90% Variance
Cumulative Explained Variance (%)
50
60
70
80
90
100
0
5
10
15
20
25
30
Number of Components

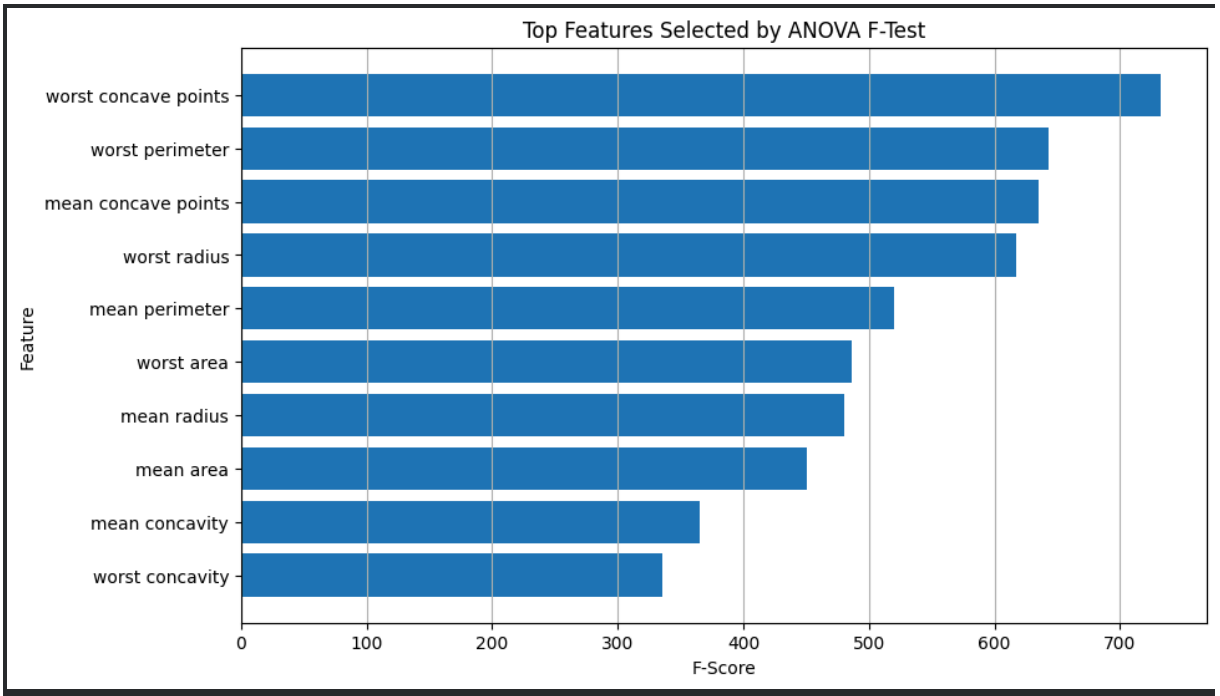


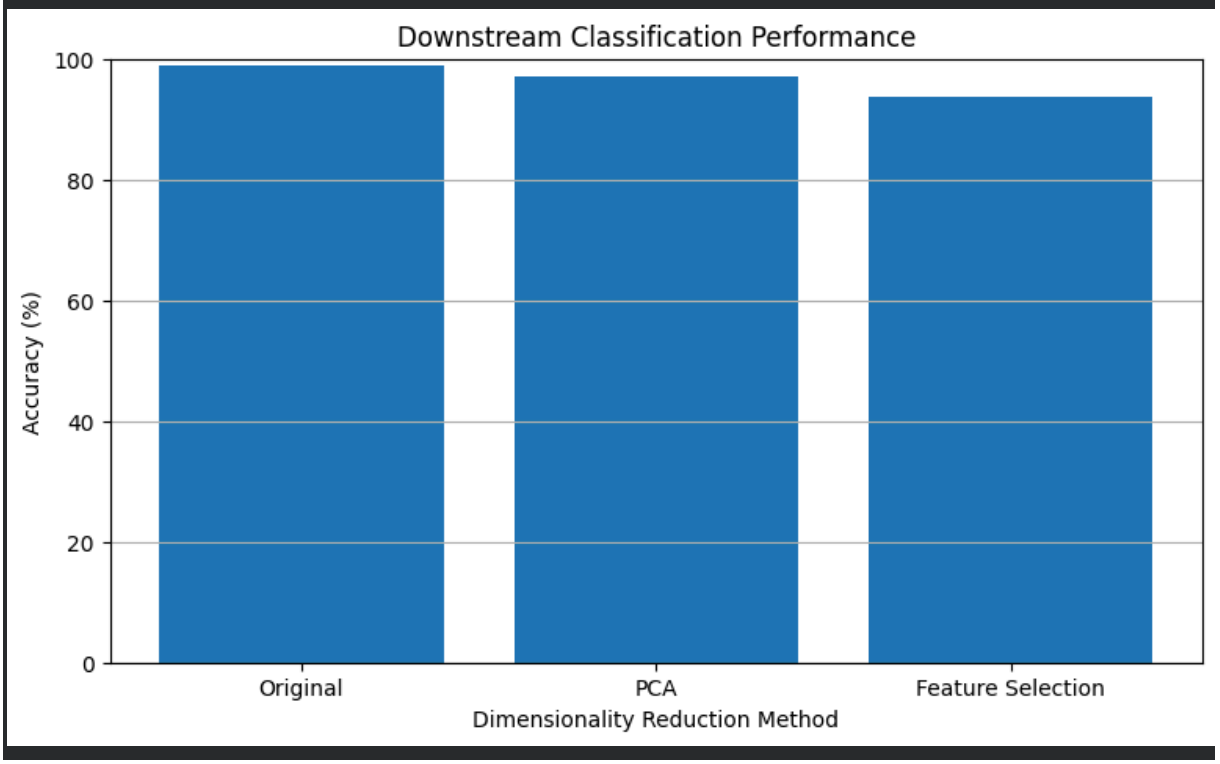
==========

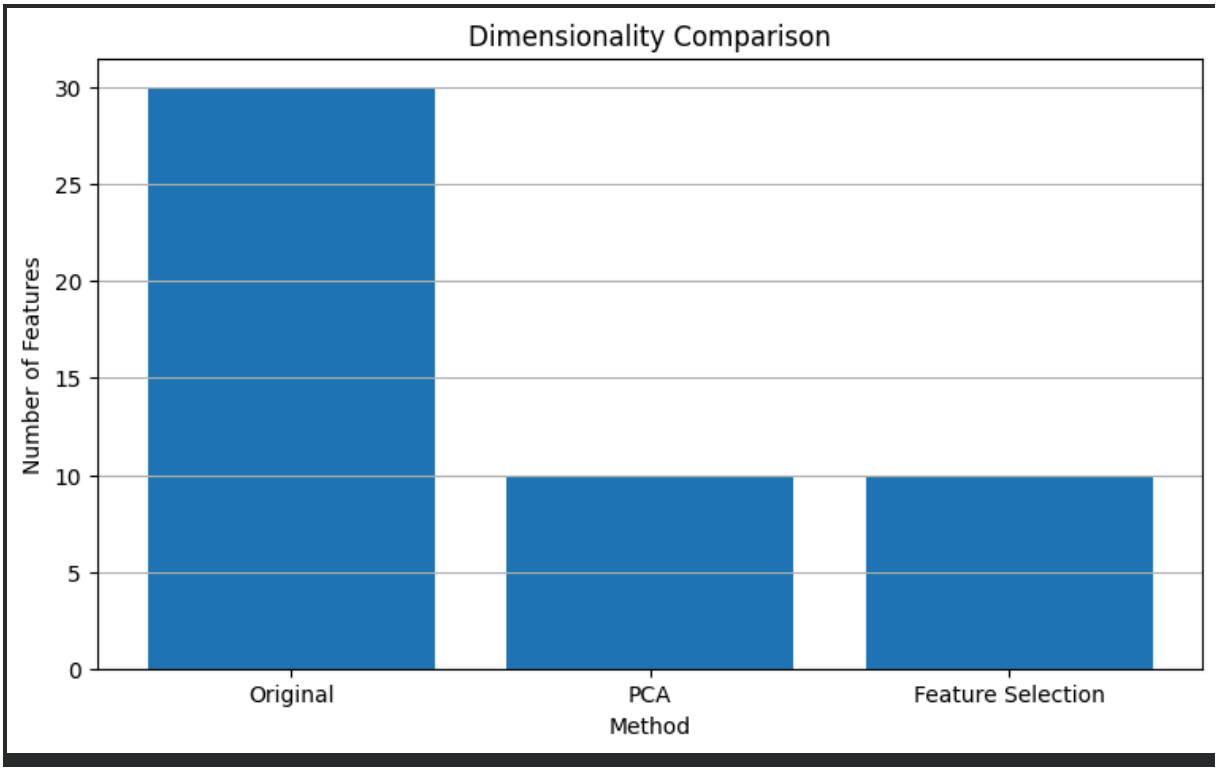
FEATURE SELECTION RANKING

==========

Feature	F-Score
worst concave points	732.283
worst perimeter	643.460
mean concave points	635.026
worst radius	617.391
mean perimeter	519.551
worst area	486.344
mean radius	480.594
mean area	450.499
mean concavity	365.481
worst concavity	335.294


Top Features Selected by ANOVA F-Test
worst concave points
worst perimeter
mean concave points
worst radius
mean perimeter
worst area
mean radius
mean area
mean concavity
worst concavity
Feature
0
100
200
300
400
500
600
700
F-Score


Downstream Classification Performance
100
80
60
40
20
0
Accuracy (%)
Original
PCA
Feature Selection
Dimensionality Reduction Method



==========

ORIGINAL FEATURES - CLASSIFICATION REPORT

==========

	precision	recall	f1-score	support
0	0.98	0.98	0.98	64
1	0.99	0.99	0.99	107
accuracy			0.99	171
macro avg	0.99	0.99	0.99	171
weighted avg	0.99	0.99	0.99	171

==========

PCA - CLASSIFICATION REPORT

==========

	precision	recall	f1-score	support
0	0.94	0.98	0.96	64
1	0.99	0.96	0.98	107
accuracy			0.97	171
macro avg	0.97	0.97	0.97	171
weighted avg	0.97	0.97	0.97	171

==========

FEATURE SELECTION - CLASSIFICATION REPORT

==========

	precision	recall	f1-score	support
0	0.92	0.91	0.91	64
1	0.94	0.95	0.95	107
accuracy			0.94	171
macro avg	0.93	0.93	0.93	171
weighted avg	0.94	0.94	0.94	171

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**15**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**20**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand